INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training

LAB REPORT

Network Traffic Analysis with Wireshark and TShark:

Investigation of the Nitroba University Harassment Scenario

Field	Detail
Lab Title	Network Traffic Analysis with Wireshark and TShark
Student Name	Fuseini Imoru Kantuogaa
Student ID	C11/26/DFIT/17291
Course Name	Case Study 2 — Email Harassment: Wireshark and TShark Attribution Investigation
Instructor Name	Mr. Amin Iddrisu
Date of Submission	17/09/2026
Version	Version 1.0

1. Executive Summary

This lab examined a recorded network capture from a simulated harassment investigation and, alongside it, developed practical competence with the command-line packet analyser TShark. The investigative element used the publicly available Nitroba University scenario, in which a member of teaching staff received anonymous threatening messages and the university provided a packet capture of the dormitory network for analysis.

Two evidence files were handled. The primary exhibit, nitroba.pcap, was acquired from the Digital Corpora archive, hash-verified, and examined in both the Wireshark graphical interface and TShark. A second, much smaller capture was used to build and rehearse the filtering, field-extraction and regular-expression techniques that were then applied to the main exhibit.

The analysis established that the anonymous messages were sent through the web service WillSelfDestruct.com by a single host on the dormitory network, 192.168.15.4, using an Apple device with the hardware address 00:17:f2:e2:c0:ce. Although the mail service concealed the sender, the same device leaked persistent Google session cookies in unencrypted HTTP traffic, tying the anonymous submission to the personal account jcoachj@gmail.com. The harassing message, recovered in full from frame 83601, was addressed to the complainant and was sent at 02:04:24 (-0400) on 22 July 2008. The investigation therefore succeeded in attributing an ostensibly anonymous communication to a named account.

2. Lab Objectives

The objectives of this lab were as follows:

- To acquire a network capture from an external evidence repository and establish its integrity through cryptographic hashing before analysis begins.
- To demonstrate competence with Wireshark display filters in isolating relevant traffic from a capture of approximately 94,000 packets.
- To develop equivalent competence with TShark, including live capture, capture filters, display filters, field extraction, protocol statistics, stream reassembly and the use of regular expressions against packet dissections.

- To recover the content of an anonymous web-based email submission from unencrypted HTTP traffic, including recipient, subject, message body and timestamp.
- To attribute that submission to a specific physical device, and then to a specific named account, by correlating link-layer, network-layer and application-layer artefacts.
- To document the investigative process to a standard that would support a coherent account of how each conclusion was reached.

3. Tools and Resources Used

All analysis was performed on a Kali Linux virtual machine. No commercial tooling was required.

Tool / Resource	**Purpose and relevance**
Kali Linux (VMware guest)	Analysis platform hosting all tools used in this lab.
Wireshark	Graphical protocol analyser used for the initial survey of nitroba.pcap, for interactive display filtering, and for inspecting reassembled HTTP payloads.
TShark	Command-line counterpart to Wireshark. Used for live capture, scripted filtering, field extraction, protocol hierarchy and IO statistics, stream following, and piping dissections into text-processing tools.
tcpdump-style capture filters (BPF)	Applied at capture time with the -f option to restrict collection by network, host or address.
wget	Retrieval of the nitroba.pcap evidence file and the basic.log sample capture.
OpenSSL (openssl dgst -md5)	Generation and re-verification of the MD5 hash used to demonstrate evidence integrity.

Tool / Resource	Purpose and relevance
grep (GNU, with -E and -P)	Pattern matching against verbose TShark output, including extended and Perl-compatible regular expressions.
regex101.com	Construction and validation of the regular expressions applied to packet dissections.
ping	Generation of controlled ICMP traffic to verify that capture filters behaved as expected.
Firefox	Generation of controlled HTTP traffic for the ubalt.pcap capture exercise.
nitroba.pcap (Digital Corpora)	Primary evidence file: 56,180,821 bytes, 94,410 packets, MD5 9981827f11968773ff815e39f5458ec8.
basic.log	Ten-packet sample capture (1,812 bytes) used to rehearse filter syntax safely before applying it to the evidence.

4. Methodology

The work was carried out in four phases. Phase one acquired and verified the evidence. Phase two surveyed the evidence in the graphical interface and identified the relevant traffic. Phase three built command-line competence against a small, disposable capture. Phase four returned to the evidence file and reproduced and extended the findings from the command line, where the results could be captured as text rather than as screenshots of a window.

4.1 Phase One — Acquisition and Integrity Verification

1. Escalated to root and created a dedicated working directory, /home/kali/harass, so that evidence remained separated from other lab material (Figure 1).

2. Retrieved nitroba.pcap from the Digital Corpora 2008-nitroba scenario archive using wget. The transfer completed with 56,180,821 bytes received (Figures 1 and 2).
3. Confirmed the file size with ls -l and generated an MD5 hash of the acquired file using openssl dgst -md5, recording the value 9981827f11968773ff815e39f5458ec8 as the acquisition hash (Figure 2).
4. Re-verified the same hash at the start of phase four, after the file had been opened repeatedly in Wireshark, to confirm that analysis had not altered the evidence (Figure 69).

4.2 Phase Two — Graphical Survey and Identification in Wireshark

5. Opened nitroba.pcap in Wireshark and confirmed the capture loaded in full, ending at packet 94,410 (Figure 3).
6. Filtered for outbound web requests to the anonymous mail service with http.request.method == "GET" && http.host == "www.willselfdestruct.com", which isolated a GET to /secure/submit from 192.168.15.4 (Figure 4).
7. Located the corresponding server responses with the content filter data-text-lines contains "secure anonymous E-mail", returning two HTTP 200 responses from 69.25.94.22 (Figure 5).
8. Examined the reassembled HTML of the first response to confirm the identity and function of the service (Figures 6 and 7).
9. Filtered for the submission itself using (ip.src == 192.168.15.4) && (ip.dst == 69.25.94.22) && http.request.method == "POST", isolating frame 83601 and exposing the URL-encoded form fields in the packet detail pane (Figures 8 and 10).
10. Inspected the Ethernet layer of frame 83601 to record the source hardware address, 00:17:f2:e2:c0:ce, identified by Wireshark as an Apple device (Figure 9).
11. Pivoted from the IP address to the hardware address with eth.addr == 00:17:f2:e2:c0:ce. This returned 73,246 of 94,410 packets, confirming that the device was a heavy user of the network and that its traffic could be followed across any change of IP address (Figure 11).
12. Searched that device's traffic for identity-bearing artefacts using eth.src == 00:17:f2:e2:c0:ce && http.cookie contains "@", which exposed unencrypted Google session cookies (Figure 12).

13. Confirmed the submission succeeded by examining the responses to /secure/submit and /secure/success (Figures 13 and 14).

4.3 Phase Three — Building TShark Competence

14. Reviewed the tshark manual page and enumerated the available capture interfaces with tshark -D (Figures 15 to 17).
15. Performed a bounded live capture on eth0 using -c to limit packet count and -w to write the output, then read the file back with -r (Figure 18).
16. Applied capture filters at collection time: first a network filter, "net 192.168.196.0/24" (Figure 19), then a source-host filter, "ip src host 192.168.196.2" (Figure 20).
17. Verified the source-host filter by generating known ICMP traffic with ping -c 10 in a second terminal and confirming that the capture contained exactly the expected echo replies (Figures 21 and 22).
18. Retrieved the sample capture basic.log and opened it in both Wireshark and TShark to confirm that the two tools present the same evidence identically (Figures 23 to 25).
19. Produced summary statistics: a protocol hierarchy with -qz io,phs, and frame counts with -qz io,stat, including a filtered second column (Figures 26 to 28).
20. Extracted individual fields in tabular form with -T fields -e, using frame.time, frame.number, ip.src, ip.dst, ip.len and eth.addr (Figures 29, 30 and 40).
21. Practised display-filter syntax: equality, inequality, field presence, the contains operator, compound && expressions, and the matches operator for regular expressions (Figures 31 to 33).
22. Examined a single frame in full with -V, walking the frame, Ethernet, IP, TCP and HTTP layers down to the decoded HTML body, and compared this with the raw hex dump produced by -x (Figures 34 to 39, 41 to 44).
23. Combined TShark with grep to highlight patterns in verbose output, including literal phrases, HTML tags, User-Agent values, URLs, dotted-quad IP addresses and email addresses (Figures 45 to 49).
24. Reassembled a complete TCP conversation with -z follow,tcp,ascii,0 (Figures 50 and 51).

25. Enumerated the available protocols and field names with tshark -G protocols and tshark -G fields, filtering the dictionary with extended and Perl-compatible regular expressions to find valid filter fields (Figures 52 to 63).

26. Used regex101.com to construct and test the expressions before applying them (Figure 64).

27. Completed the phase with a controlled end-to-end exercise: captured 500 packets to ubalt.pcap using the capture filter "host 204.52.129.211", generated the traffic by loading the corresponding website in Firefox, then recovered a telephone number and both hardware addresses from the capture using regular expressions (Figures 65 to 68).

4.4 Phase Four — Command-Line Analysis of the Evidence

28. Returned to the evidence directory and re-verified the MD5 hash against the value recorded at acquisition (Figure 69).

29. Established the temporal boundaries of the capture by reading the first and last frames, and confirmed the total at 94,410 frames over a duration of 15,760 seconds, approximately four hours and twenty-three minutes (Figures 70 and 71).

30. Reproduced the willselfdestruct.com GET requests from the command line, then re-ran the query with -T fields to obtain timestamps, inter-packet deltas and request URIs in tabular form. This exposed the full sequence of the submission, from /secure/submit through the image requests to /secure/success (Figures 72 and 73).

31. Isolated the POST in frame 83601 and confirmed the recipient field, then extracted the complete URL-encoded form payload together with its absolute timestamp (Figures 74 and 75).

32. Recorded the source hardware address of the POST and quantified that device’s presence in the capture: 73,246 frames totalling 44,742,268 bytes (Figures 76 and 77).

33. Swept the device’s traffic for identity artefacts using an email-address regular expression against the verbose dissection, recovering the Google session cookies and the associated account (Figures 78 and 79).

34. Recovered the remaining correlating identifiers from the same traffic, including the spoofed sender address supplied to the mail service and a concurrent instant-messenger login (Figure 80).

35. Recorded the absolute timestamp of the harassing POST as the final item of the evidential timeline (Figure 81).

5. Screenshots and Evidence

The figures below are presented in the order in which the work was performed. Figures 1 to 14 cover acquisition and the graphical analysis of the evidence. Figures 15 to 68 cover the TShark competence exercises. Figures 69 to 81 cover the command-line analysis of the evidence file.

5.1 Acquisition and Graphical Analysis (Figures 1–14)

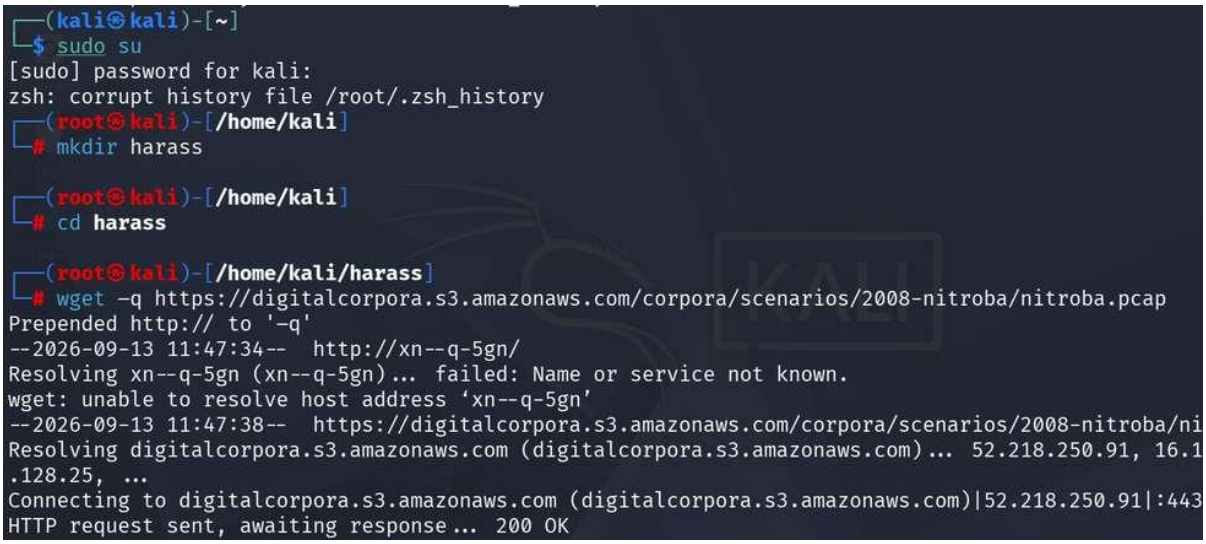


Figure 1: *Escalation to root, creation of the /home/kali/harass working directory, and acquisition of nitroba.pcap from the Digital Corpora 2008-nitroba scenario archive using wget.*

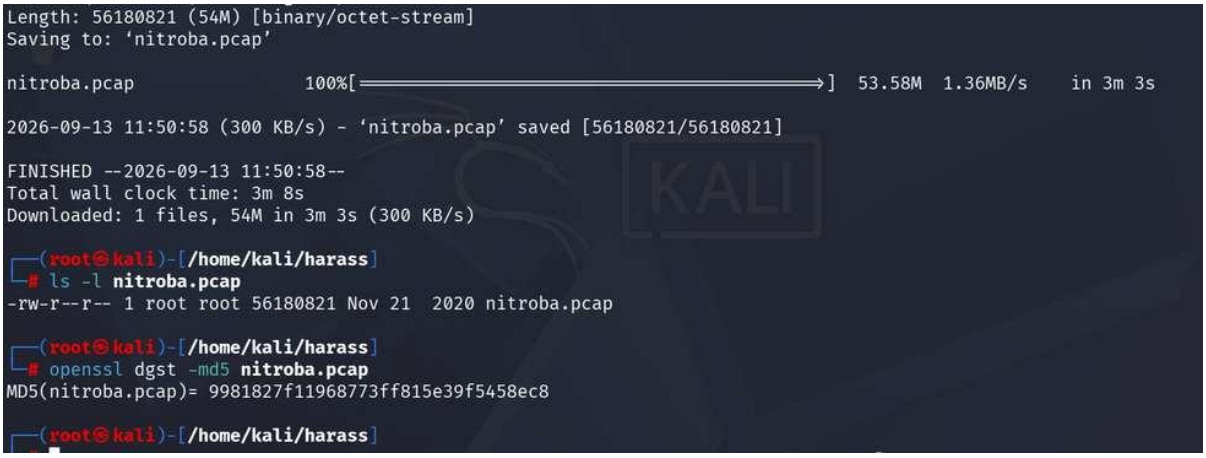


Figure 2: *Completion of the download (56,180,821 bytes), directory listing confirming file size, and generation of the MD5 acquisition hash 9981827f11968773ff815e39f5458ec8 using openssl dgst.*

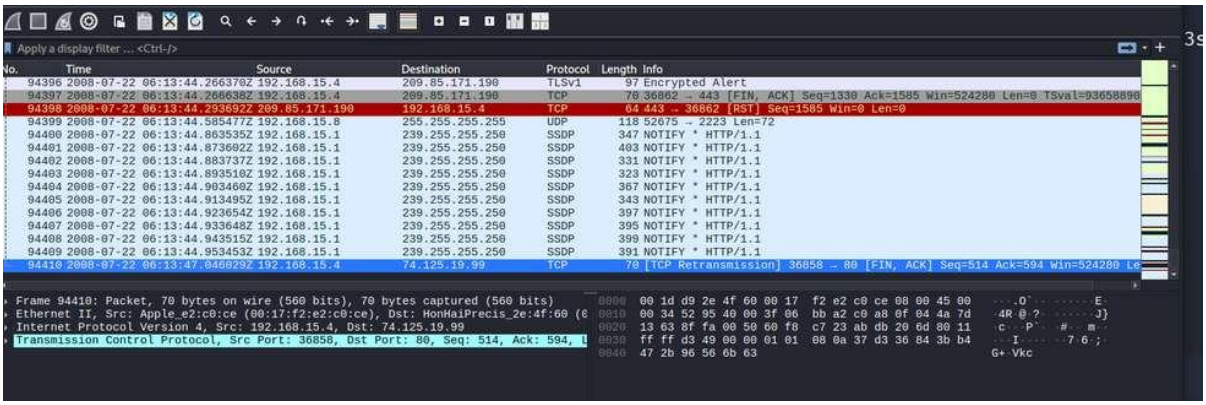


Figure 3: *The unfiltered nitroba.pcap capture opened in Wireshark, showing the tail of the capture at 94,410 packets and confirming the evidence file loaded without error.*

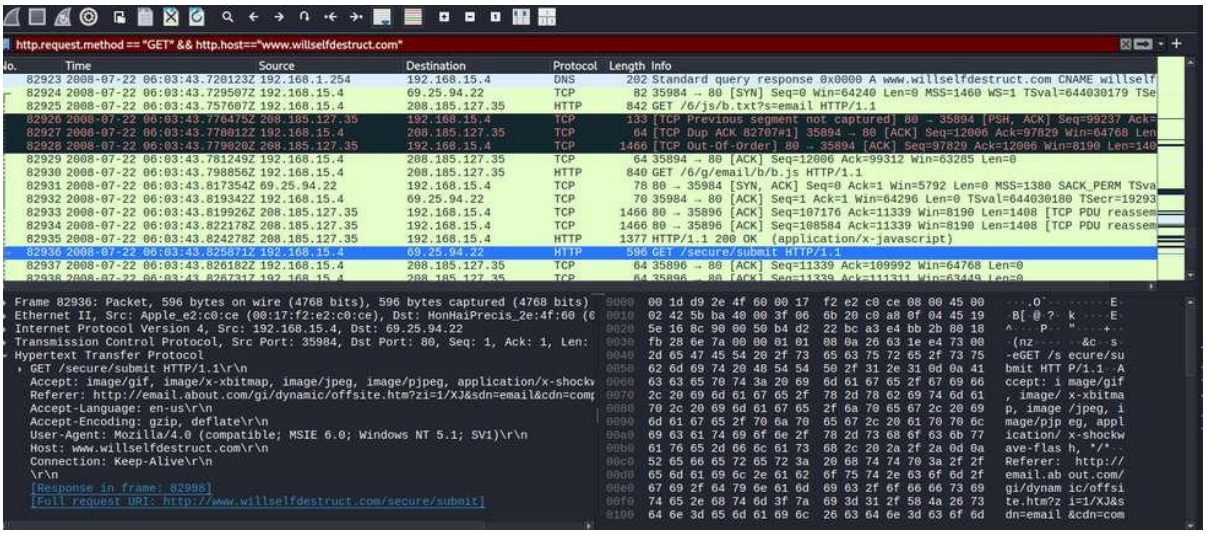


Figure 4: *Display filter http.request.method == "GET" && http.host == "www.willselfdestruct.com" isolating frame 82936, a GET request to /secure/submit issued by host 192.168.15.4.*

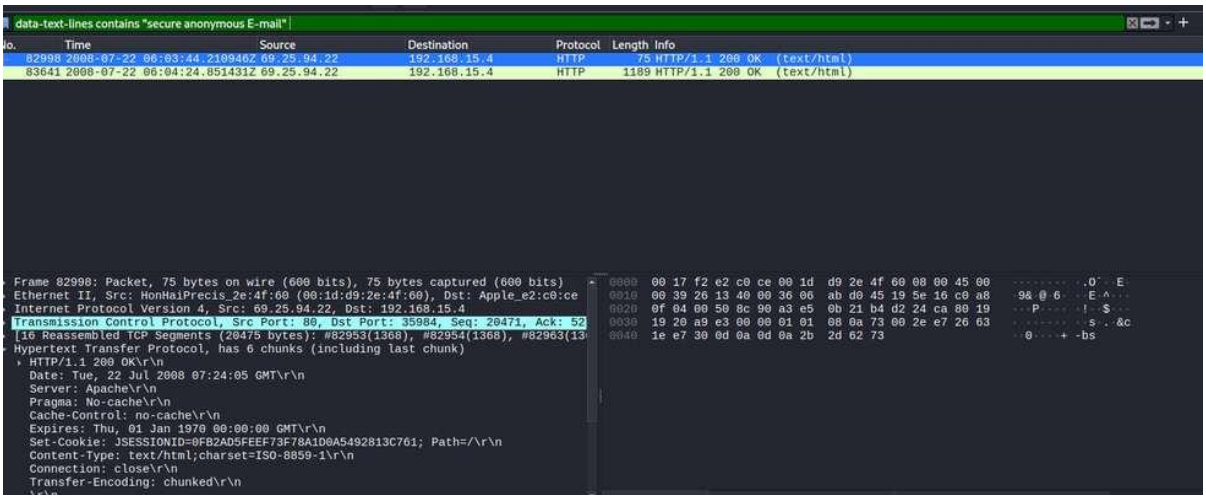


Figure 5: *Display filter data-text-lines contains "secure anonymous E-mail" returning two HTTP 200 responses from 69.25.94.22 to 192.168.15.4, at 06:03:44 and 06:04:24 UTC.*

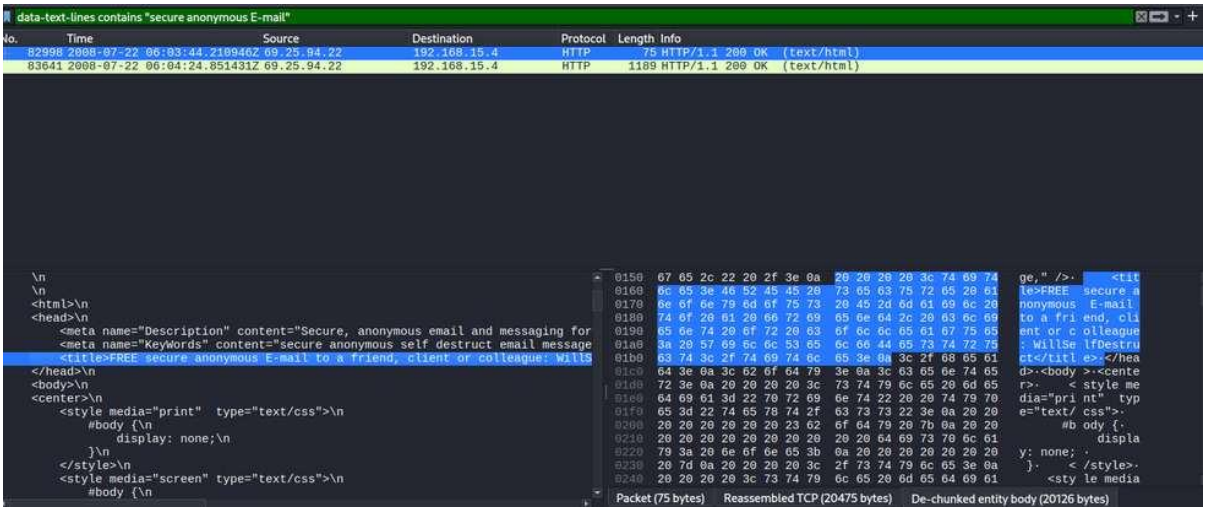


Figure 6: *The reassembled HTML body of frame 82998, whose title identifies the service as WillSelfDestruct, a free anonymous self-destructing email provider.*

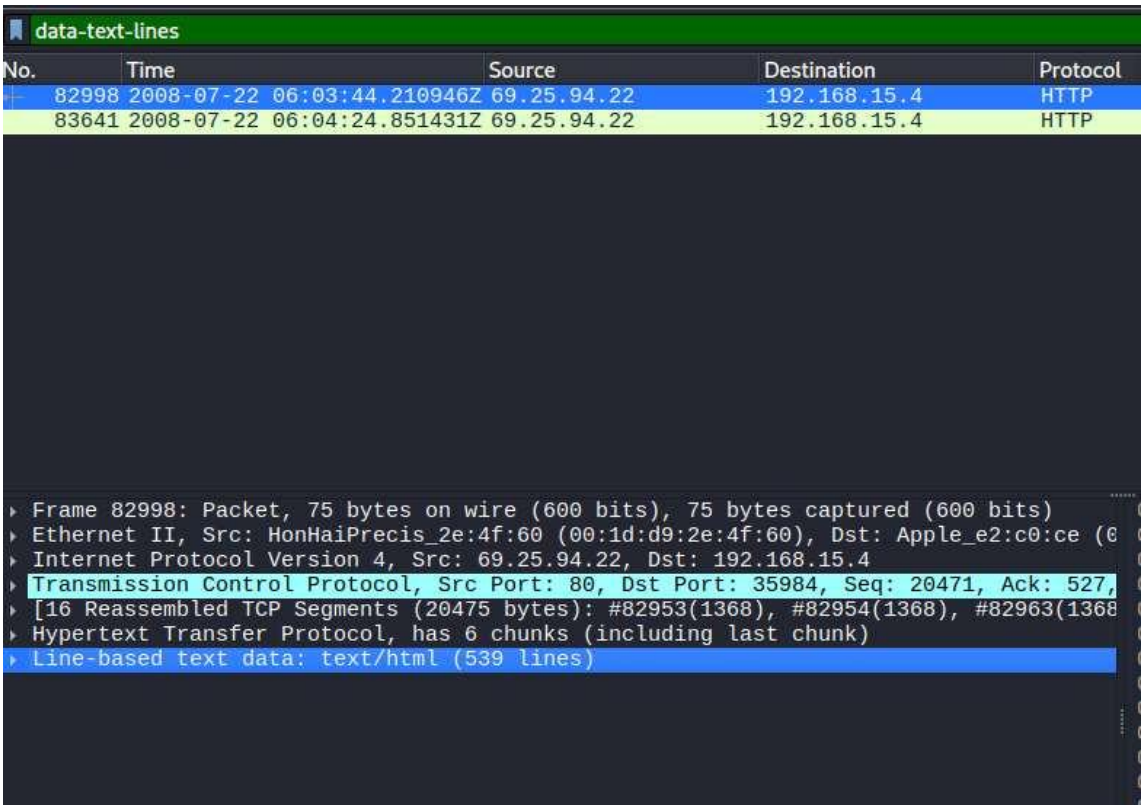


Figure 7: *Packet detail tree for frame 82998, showing 16 reassembled TCP segments totalling 20,475 bytes and a line-based text payload of 539 lines.*

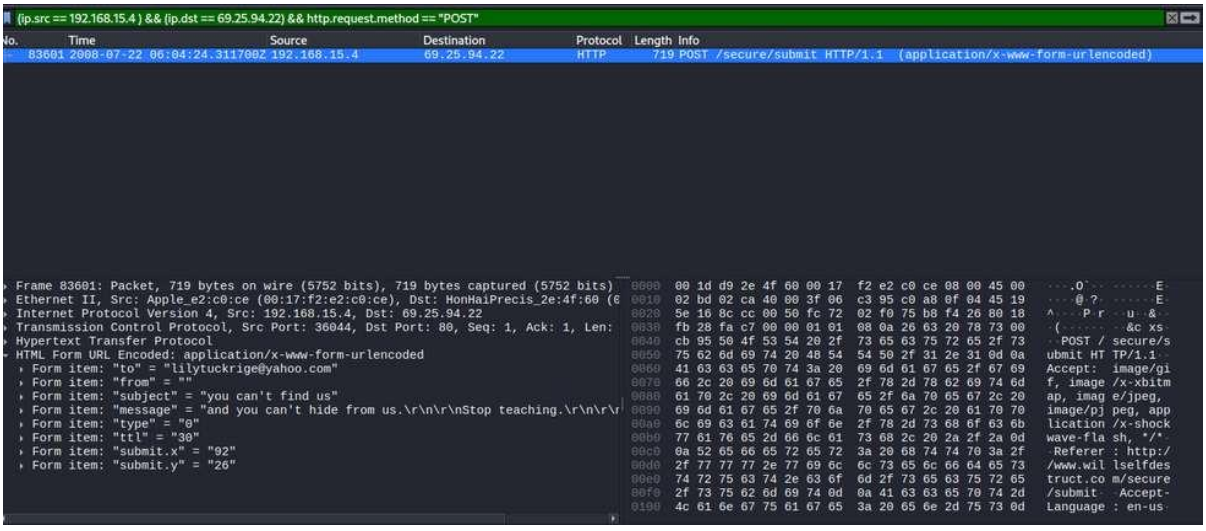


Figure 8: *Display filter (ip.src == 192.168.15.4) && (ip.dst == 69.25.94.22) && http.request.method == "POST" isolating frame 83601, the HTTP POST that submitted the harassing message.*

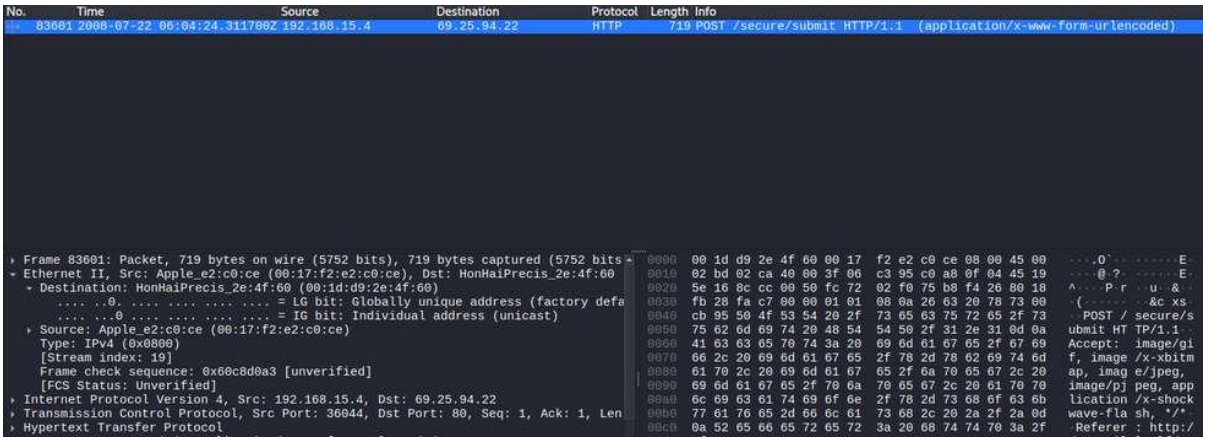


Figure 9: *Ethernet layer of frame 83601, identifying the source hardware address as Apple_e2:c0:ce (00:17:f2:e2:c0:ce) and the destination as the HonHaiPrecis gateway.*

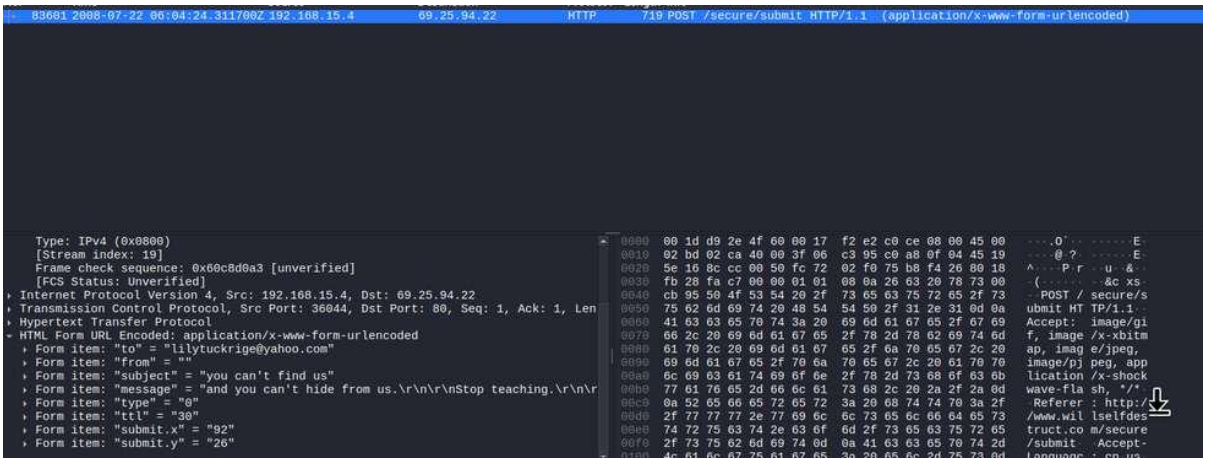


Figure 10: *Expanded URL-encoded form of frame 83601, showing the recipient, the empty sender field, the subject line and the message body submitted through the anonymous mailer.*

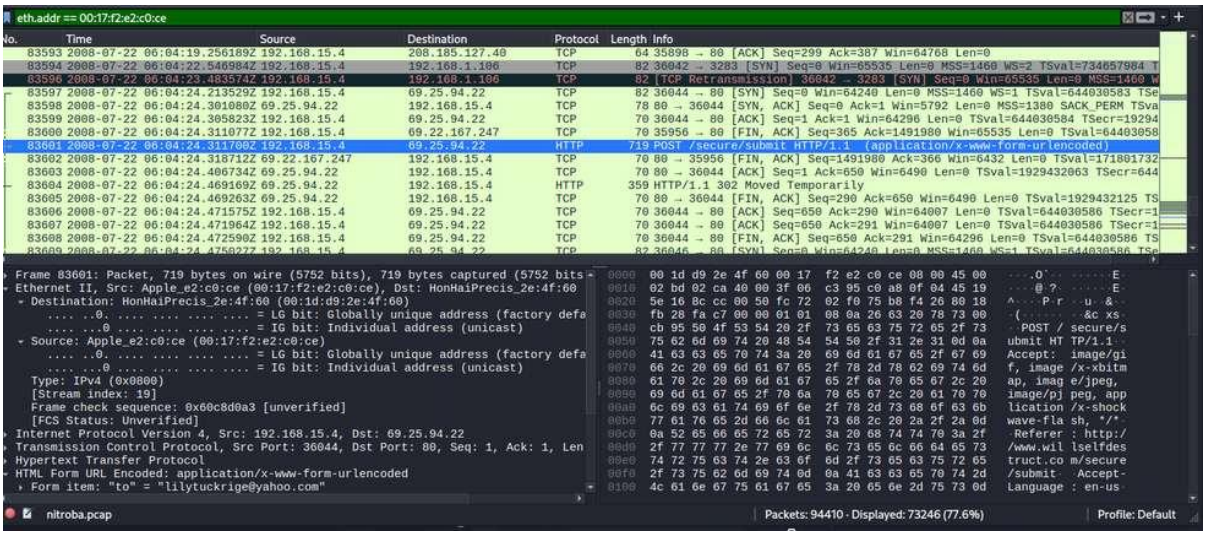


Figure 11: *Display filter eth.addr == 00:17:f2:e2:c0:ce reducing the capture to 73,246 of 94,410 packets (77.6%), establishing that a single host generated most of the traffic.*

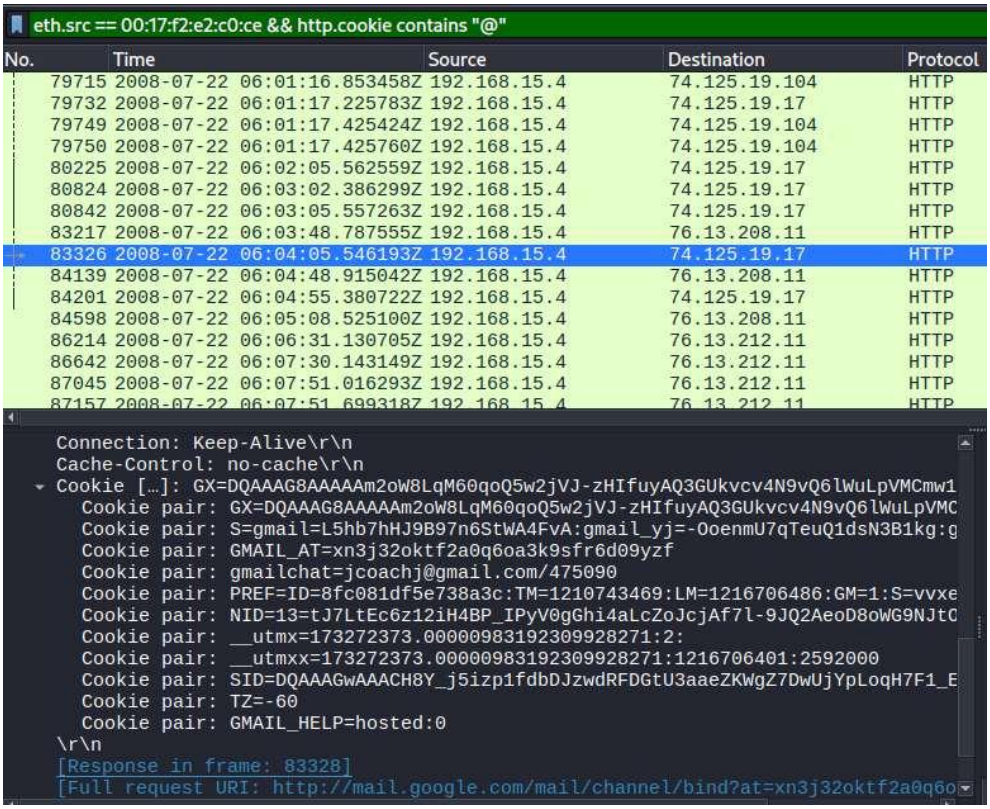


Figure 12: *Display filter eth.src == 00:17:f2:e2:c0:ce && http.cookie contains "@" exposing Google session cookies, including the gmailchat cookie carrying the account jcoachj@gmail.com.*

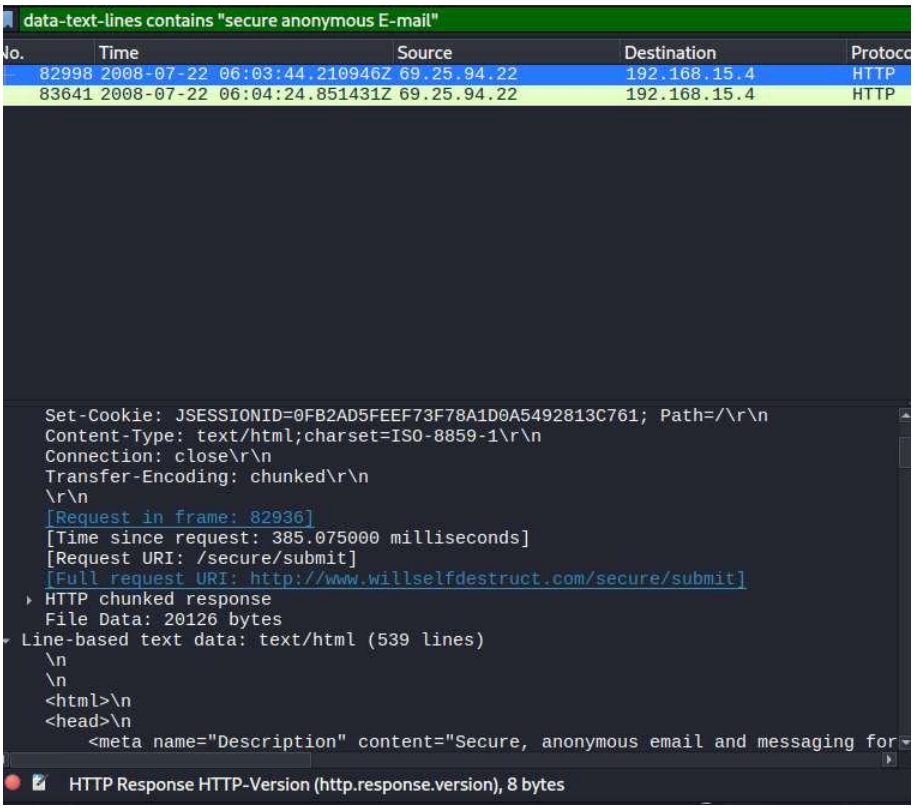


Figure 13: *HTTP response to the /secure/submit request, 20,126 bytes of chunked text/html returned 385 ms after the request.*

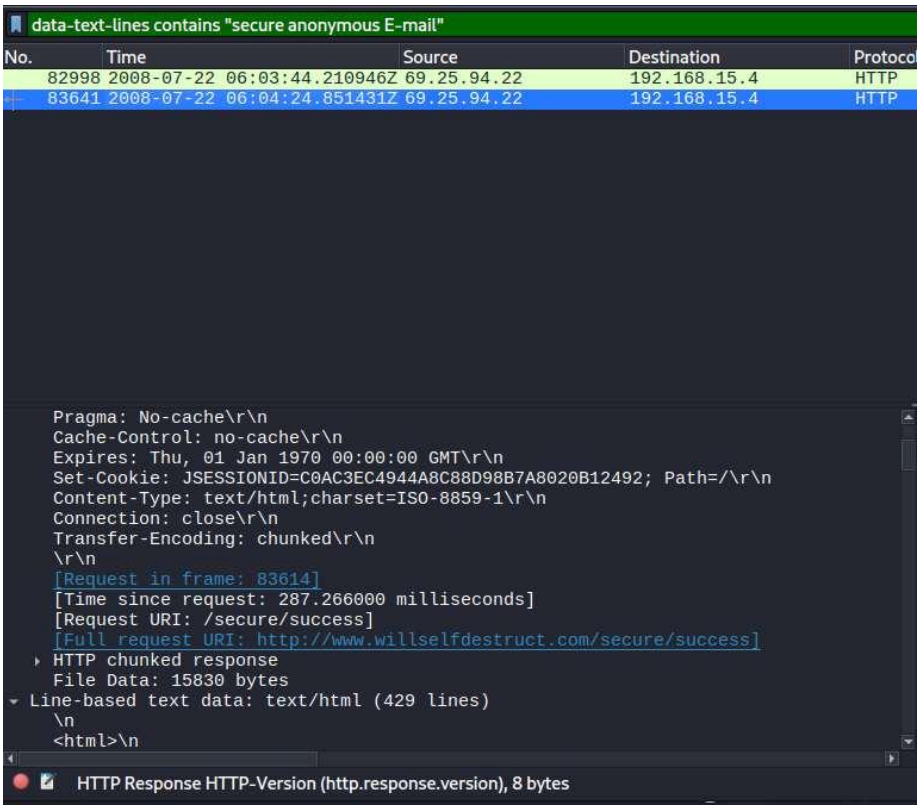


Figure 14: *HTTP response to /secure/success, 15,830 bytes, confirming the anonymous message was accepted and delivered by the service.*

5.2 TShark Fundamentals (Figures 15–68)

Capture, interfaces and capture filters



Figure 15: *Invocation of the tshark manual page at the start of the command-line analysis exercise.*

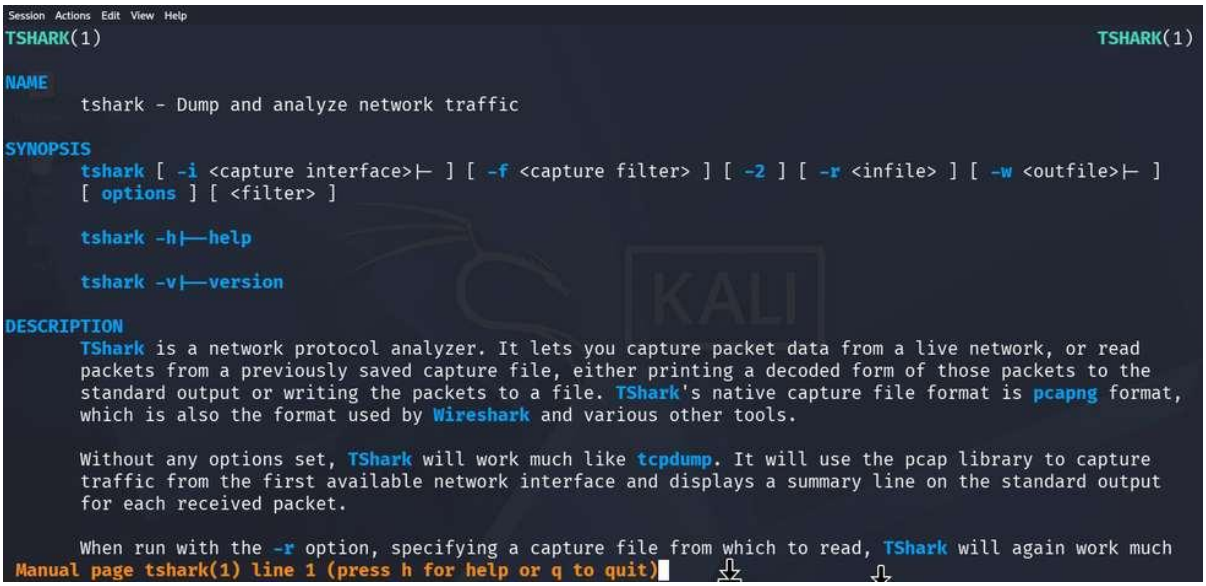


Figure 16: *The tshark(1) manual page, documenting the -i, -f, -r, -w and filter options used throughout this lab.*

```
┌──(kali㉿kali)-[~]
└─$ tshark -D
1. eth0
2. any
3. lo (Loopback)
4. docker0
5. bluetooth-monitor
6. nflog
7. nfqueue
8. dbus-system
9. dbus-session
10. ciscodump (Cisco remote capture)
11. dpauxmon (DisplayPort AUX channel monitor capture)
12. randpkt (Random packet generator)
13. sdjournal (systemd Journal Export)
14. sshdig (SSH remote syscall capture)
15. sshdump (SSH remote capture)
16. udpdump (UDP Listener remote capture)
17. wifidump (Wi-Fi remote capture)

┌──(kali㉿kali)-[~]
└─$
```

Figure 17: *Enumeration of available capture interfaces with tshark -D, listing seventeen interfaces on the Kali host.*

```
┌──(kali㉿kali)-[~]
└─$ tshark -i eth0 -c 10 -w mycapture.pcap
Capturing on 'eth0'
10

┌──(kali㉿kali)-[~]
└─$ ll mycapture.pcap
-rw------- 1 kali kali 1296 Sep 13 20:07 mycapture.pcap

┌──(kali㉿kali)-[~]
└─$ tshark -r mycapture.pcap
    1 0.000000000 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.168.196.2? Tell 192.168.196.1
    2 0.730593510 192.168.196.128 → 162.159.200.123 NTP 90 NTP Version 4, client
    3 0.818995961 162.159.200.123 → 192.168.196.128 NTP 90 NTP Version 4, server
    4 4.729409628 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.168.196.2? Tell 192.168.196.1
    5 5.362765212 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.168.196.2? Tell 192.168.196.1
    6 6.254663255 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.168.196.2? Tell 192.168.196.1
    7 9.523979489 VMware_87:06:29 → VMware_e8:a1:b6 ARP 42 Who has 192.168.196.2? Tell 192.168.196.128
    8 9.524594842 VMware_e8:a1:b6 → VMware_87:06:29 ARP 60 192.168.196.2 is at 00:50:56:e8:a1:b6
    9 11.487271705 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.168.196.2? Tell 192.168.196.1
   10 12.104757424 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.168.196.2? Tell 192.168.196.1

┌──(kali㉿kali)-[~]
└─$
```

Figure 18: *A ten-packet live capture on eth0 written to mycapture.pcap with -c and -w, followed by reading the file back with -r; ARP and NTP traffic is visible.*

```
┌──(kali㉿kali)-[~]
└─$ tshark -f "net 192.168.196.0/24" -i eth0 -c 10 -w mycapture192_16
8_196_0.pcap
Capturing on 'eth0'
10

┌──(kali㉿kali)-[~]
└─$ tshark -r mycapture192_168_196_0.pcap
    1 0.000000000 192.168.196.128 → 162.159.200.123 NTP 90 NTP Versio
n 4, client
    2 0.043752338 VMware_e8:a1:b6 → Broadcast    ARP 60 Who has 192.1
68.196.128? Tell 192.168.196.2
    3 0.043768873 VMware_87:06:29 → VMware_e8:a1:b6 ARP 42 192.168.19
6.128 is at 00:0c:29:87:06:29
    4 0.043990632 162.159.200.123 → 192.168.196.128 NTP 90 NTP Versio
n 4, server
    5 8.663713051 VMware_87:06:29 → VMware_e8:a1:b6 ARP 42 Who has 19
2.168.196.2? Tell 192.168.196.128
    6 8.664024385 VMware_e8:a1:b6 → VMware_87:06:29 ARP 60 192.168.19
6.2 is at 00:50:56:e8:a1:b6
    7 18.014601882 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.
168.196.2? Tell 192.168.196.1
    8 18.964812042 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.
168.196.2? Tell 192.168.196.1
    9 19.815142396 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.
168.196.2? Tell 192.168.196.1
   10 20.713709763 VMware_c0:00:08 → Broadcast    ARP 60 Who has 192.
168.196.2? Tell 192.168.196.1

┌──(kali㉿kali)-[~]
└─$
```

Figure 19: *Capture filter "net 192.168.196.0/24" restricting collection to the lab subnet, with the resulting file read back to verify the filter applied at capture time.*

```
┌──(kali㉿kali)-[~]
└─$ sudo tshark -i eth0 -f "ip src host 192.168.196.2" -c 10 -w /tmp/
mycapture192_168_196_2.pcap
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
10

┌──(kali㉿kali)-[~]
└─$ ls -lh /tmp/mycapture192_168_196_2.pcap
-rw------- 1 root root 1.7K Sep 14 09:42 /tmp/mycapture192_168_196_2.
pcap
```

Figure 20: *Capture filter "ip src host 192.168.196.2" run under sudo, writing 10 packets to /tmp/mycapture192_168_196_2.pcap.*

```
┌──(kali㉿kali)-[~]
└─$ sudo tshark -r /tmp/mycapture192_168_196_2.pcap
Running as user "root" and group "root". This could be dangerous.
    1 0.000000000 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=1/256, ttl=128
    2 1.020732052 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=2/512, ttl=128
    3 2.040696259 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=3/768, ttl=128
    4 3.068561756 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=4/1024, ttl=128
    5 4.089113458 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=5/1280, ttl=128
    6 5.089549595 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=6/1536, ttl=128
    7 6.090810262 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=7/1792, ttl=128
    8 7.096778357 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=8/2048, ttl=128
    9 8.098111089 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=9/2304, ttl=128
   10 9.098866628 192.168.196.2 → 192.168.196.128 ICMP 98 Echo (ping)
 reply    id=0×c24e, seq=10/2560, ttl=128

┌──(kali㉿kali)-[~]
└─$
```

Figure 21: *Reading the filtered capture: all ten frames are ICMP echo replies from 192.168.196.2, confirming the source-host capture filter worked as intended.*

```
┌──(kali㉿kali)-[~]
└─$ ping -c 10 192.168.196.2
PING 192.168.196.2 (192.168.196.2) 56(84) bytes of data.
64 bytes from 192.168.196.2: icmp_seq=1 ttl=128 time=0.193 ms
64 bytes from 192.168.196.2: icmp_seq=2 ttl=128 time=0.330 ms
64 bytes from 192.168.196.2: icmp_seq=3 ttl=128 time=0.386 ms
64 bytes from 192.168.196.2: icmp_seq=4 ttl=128 time=0.360 ms
64 bytes from 192.168.196.2: icmp_seq=5 ttl=128 time=0.595 ms
64 bytes from 192.168.196.2: icmp_seq=6 ttl=128 time=0.714 ms
64 bytes from 192.168.196.2: icmp_seq=7 ttl=128 time=0.466 ms
64 bytes from 192.168.196.2: icmp_seq=8 ttl=128 time=0.518 ms
64 bytes from 192.168.196.2: icmp_seq=9 ttl=128 time=0.739 ms
64 bytes from 192.168.196.2: icmp_seq=10 ttl=128 time=0.386 ms

── 192.168.196.2 ping statistics ──
10 packets transmitted, 10 received, 0% packet loss, time 10110ms
rtt min/avg/max/mdev = 0.193/0.468/0.739/0.165 ms

┌──(kali㉿kali)-[~]
```

Figure 22: *The ping -c 10 command issued in a second terminal that generated the ICMP traffic captured in Figure 21; all ten packets were returned with zero loss.*

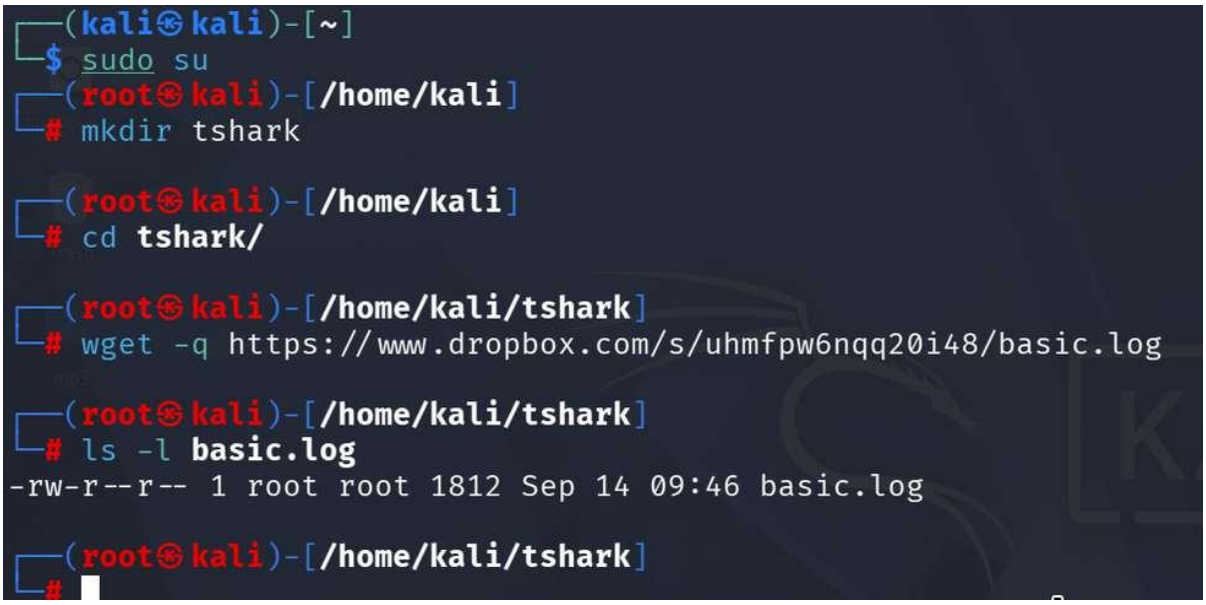


Figure 23: *Creation of the /home/kali/tshark working directory and retrieval of the sample capture basic.log (1,812 bytes) for the display-filter exercises.*

Reading captures and producing statistics

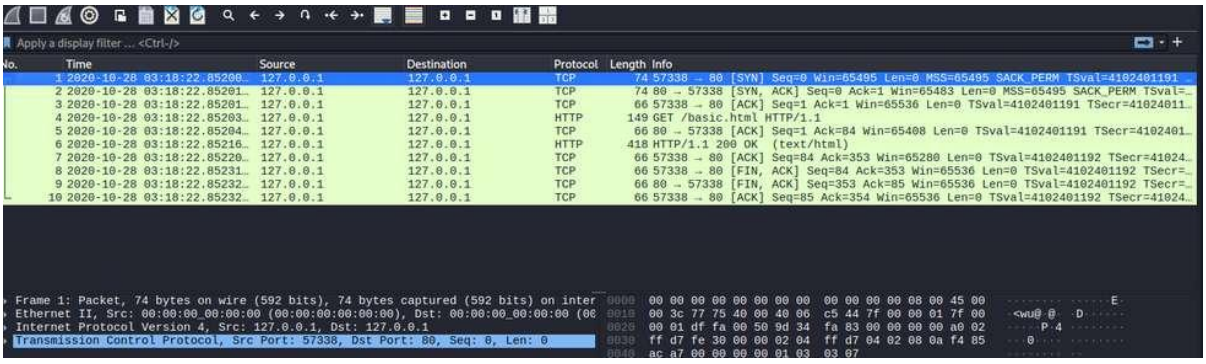


Figure 24: *basic.log opened in the Wireshark GUI, showing a complete ten-packet loopback HTTP transaction between 127.0.0.1:57338 and 127.0.0.1:80.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log
Running as user "root" and group "root". This could be dangerous.
    1 0.000000000    127.0.0.1 → 127.0.0.1    TCP 74 57338 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM
TSval=4102401191 TSecr=0 WS=128
    2 0.000008171    127.0.0.1 → 127.0.0.1    TCP 74 80 → 57338 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495
 SACK_PERM TSval=4102401191 TSecr=4102401191 WS=128
    3 0.000014783    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=41024011
91 TSecr=4102401191
    4 0.000035452    127.0.0.1 → 127.0.0.1    HTTP 149 GET /basic.html HTTP/1.1
    5 0.000042870    127.0.0.1 → 127.0.0.1    TCP 66 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408 Len=0 TSval=4102401
191 TSecr=4102401191
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)
    7 0.000202191    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280 Len=0 TSval=41024
01192 TSecr=4102401192
    8 0.000307996    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [FIN, ACK] Seq=84 Ack=353 Win=65536 Len=0 TSval=
4102401192 TSecr=4102401192
    9 0.000321475    127.0.0.1 → 127.0.0.1    TCP 66 80 → 57338 [FIN, ACK] Seq=353 Ack=85 Win=65536 Len=0 TSval=
4102401192 TSecr=4102401192
   10 0.000323997    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [ACK] Seq=85 Ack=354 Win=65536 Len=0 TSval=41024
01192 TSecr=4102401192

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 25: *The same capture read with tshark -r basic.log, demonstrating equivalence between the graphical and command-line views of the evidence.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -qz io,phs -r basic.log
Running as user "root" and group "root". This could be dangerous.

===================================================================
Protocol Hierarchy Statistics
Filter:

frame                                    frames:10 bytes:1111
  eth                                    frames:10 bytes:1111
    ip                                   frames:10 bytes:1111
      tcp                                frames:10 bytes:1111
        http                             frames:2 bytes:567
          data-text-lines                frames:1 bytes:418
===================================================================

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 26: *Protocol hierarchy statistics produced with -qz io,phs: 10 frames, of which 2 are HTTP and 1 carries line-based text data.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -qz io,stat,0,"COUNT(frame) frame"
Running as user "root" and group "root". This could be dangerous.

================================================
| IO Statistics                                |
|                                              |
| Duration: 0.000 secs                         |
| Interval: 0.000 secs                         |
|                                              |
| Col  1: COUNT(frame) frame                   |
|----------------------------------------------|
|                  |1      |                   |
| Interval         | COUNT |                   |
|------------------------------|               |
| 0.000 <> 0.000   |    10 |                   |
================================================

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 27: *IO statistics with a single COUNT(frame) column, returning a total of 10 frames over a zero-second interval.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -qz io,stat,0,"COUNT(frame)","data-text-lines contains \"first\""
Running as user "root" and group "root". This could be dangerous.

================================================
| IO Statistics                                |
|                                              |
| Duration: 0.000 secs                         |
| Interval: 0.000 secs                         |
|                                              |
| Col  1: COUNT(frame)                         |
|      2: data-text-lines contains "first"     |
|----------------------------------------------|
|                |1      |2                    |
| Interval       | COUNT | Frames | Bytes      |
|----------------------------------------------|
| 0.000 <> 0.000 |     0 |      1 |   418      |
================================================

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 28: *IO statistics with a second filtered column, counting the single frame (418 bytes) whose text payload contains the string "first".*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -T fields -e frame.time -e frame
Running as user "root" and group "root". This could be dangerous.
2020-10-27T23:18:22.852002777-0400      Frame 1: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bit
s) on interface lo, id 0
2020-10-27T23:18:22.852010948-0400      Frame 2: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bit
s) on interface lo, id 0
2020-10-27T23:18:22.852017560-0400      Frame 3: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bit
s) on interface lo, id 0
2020-10-27T23:18:22.852038229-0400      Frame 4: Packet, 149 bytes on wire (1192 bits), 149 bytes captured (1192
 bits) on interface lo, id 0
2020-10-27T23:18:22.852045647-0400      Frame 5: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bit
s) on interface lo, id 0
2020-10-27T23:18:22.852163465-0400      Frame 6: Packet, 418 bytes on wire (3344 bits), 418 bytes captured (3344
 bits) on interface lo, id 0
2020-10-27T23:18:22.852204968-0400      Frame 7: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bit
s) on interface lo, id 0
2020-10-27T23:18:22.852310773-0400      Frame 8: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bit
s) on interface lo, id 0
2020-10-27T23:18:22.852324252-0400      Frame 9: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bit
s) on interface lo, id 0
2020-10-27T23:18:22.852326774-0400      Frame 10: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bi
ts) on interface lo, id 0

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 29: *Field extraction with -T fields -e frame.time -e frame, producing machine-readable per-frame timestamps and summaries.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -T fields -e frame.number -e ip.src -e ip.dst -e ip.len
Running as user "root" and group "root". This could be dangerous.
1       127.0.0.1       127.0.0.1       60
2       127.0.0.1       127.0.0.1       60
3       127.0.0.1       127.0.0.1       52
4       127.0.0.1       127.0.0.1       135
5       127.0.0.1       127.0.0.1       52
6       127.0.0.1       127.0.0.1       404
7       127.0.0.1       127.0.0.1       52
8       127.0.0.1       127.0.0.1       52
9       127.0.0.1       127.0.0.1       52
10      127.0.0.1       127.0.0.1       52

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 30: *Field extraction of frame.number, ip.src, ip.dst and ip.len, the tabular output format most useful for scripted triage.*

Display filters

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "eth.addr ≠ 11:11:11:11:11:11"
Running as user "root" and group "root". This could be dangerous.
    1 0.000000000    127.0.0.1 → 127.0.0.1    TCP 74 57338 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM
TSval=4102401191 TSecr=0 WS=128
    2 0.000008171    127.0.0.1 → 127.0.0.1    TCP 74 80 → 57338 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495
 SACK_PERM TSval=4102401191 TSecr=4102401191 WS=128
    3 0.000014783    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=41024011
91 TSecr=4102401191
    4 0.000035452    127.0.0.1 → 127.0.0.1    HTTP 149 GET /basic.html HTTP/1.1
    5 0.000042870    127.0.0.1 → 127.0.0.1    TCP 66 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408 Len=0 TSval=4102401
191 TSecr=4102401191
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)
    7 0.000202191    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280 Len=0 TSval=41024
01192 TSecr=4102401192
    8 0.000307996    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [FIN, ACK] Seq=84 Ack=353 Win=65536 Len=0 TSval=
4102401192 TSecr=4102401192
    9 0.000321475    127.0.0.1 → 127.0.0.1    TCP 66 80 → 57338 [FIN, ACK] Seq=353 Ack=85 Win=65536 Len=0 TSval=
4102401192 TSecr=4102401192
   10 0.000323997    127.0.0.1 → 127.0.0.1    TCP 66 57338 → 80 [ACK] Seq=85 Ack=354 Win=65536 Len=0 TSval=41024
01192 TSecr=4102401192

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 31: *A negated display filter, eth.addr != 11:11:11:11:11:11, returning all ten frames and demonstrating inequality matching.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "frame.number == 6"
Running as user "root" and group "root". This could be dangerous.
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "data-text-lines"
Running as user "root" and group "root". This could be dangerous.
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)
```

Figure 32: *Two equivalent single-condition display filters: selection by frame.number and selection by the presence of the data-text-lines field, both isolating frame 6.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "data-text-lines contains \"first\""
Running as user "root" and group "root". This could be dangerous.
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "frame.number == 6 && data-text-lines contains \"first\""
Running as user "root" and group "root". This could be dangerous.
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "data-text-lines matches \"(paragraph|Head)\""
Running as user "root" and group "root". This could be dangerous.
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "http.user_agent matches \"curl\""
Running as user "root" and group "root". This could be dangerous.
    4 0.000035452    127.0.0.1 → 127.0.0.1    HTTP 149 GET /basic.html HTTP/1.1
```

Figure 33: *Progressive display-filter refinement: contains, a compound && expression, the matches regular-expression operator, and a match on http.user_agent.*

Verbose dissection and raw output

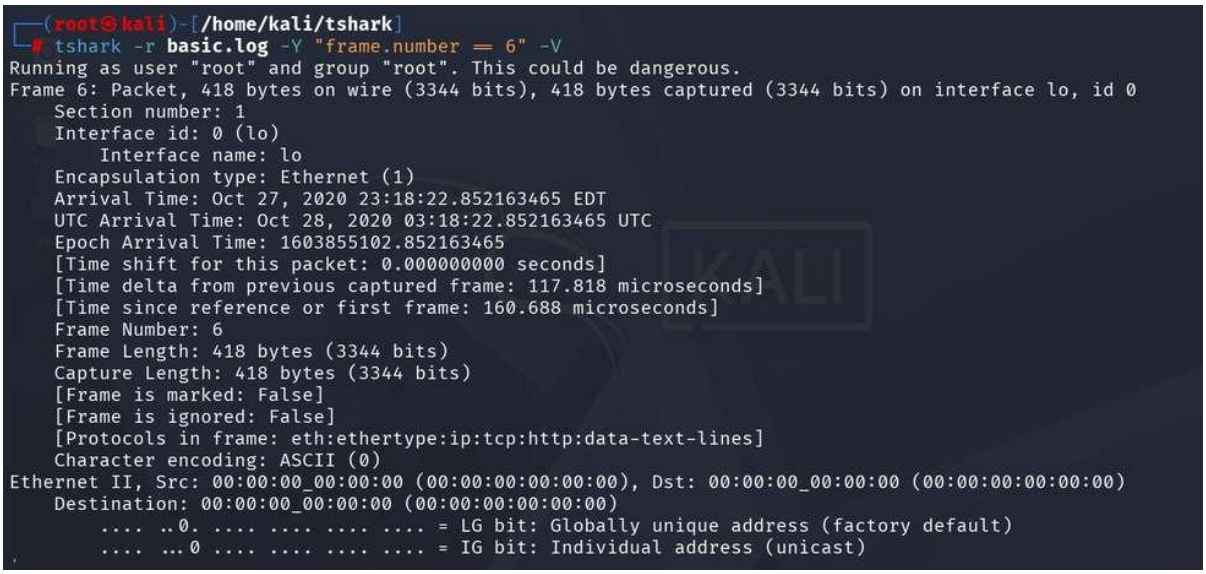


Figure 34: *Verbose output (-V) for frame 6, showing frame metadata, arrival timestamps and the protocol stack eth:ethertype:ip:tcp:http:data-text-lines.*

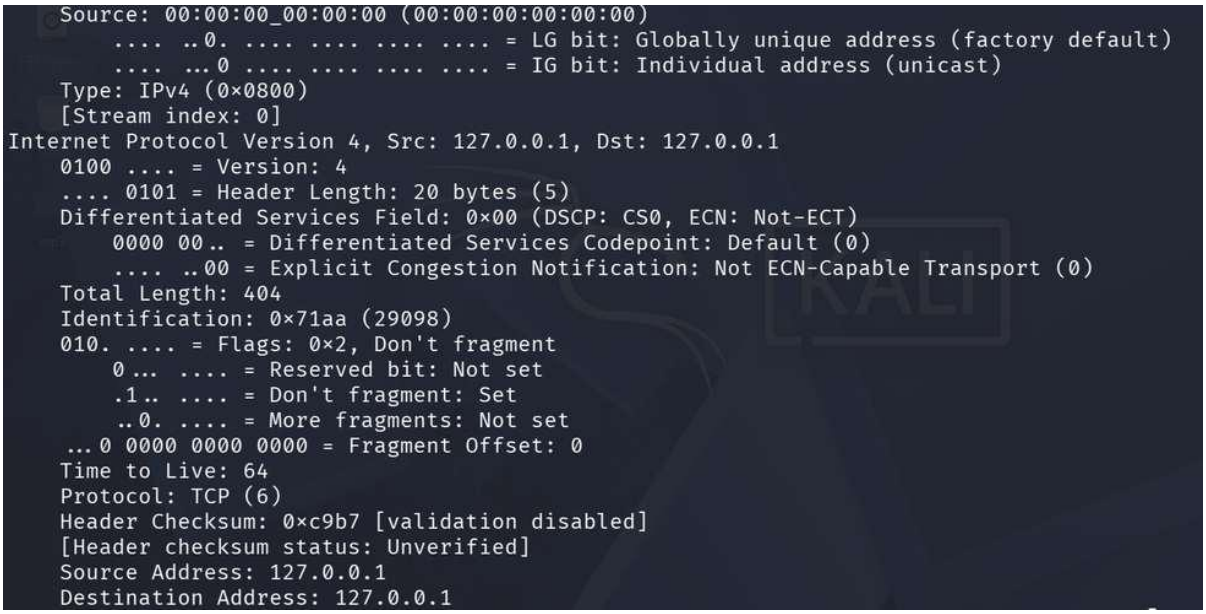


Figure 35: *Ethernet and IPv4 layers of frame 6 in verbose output, including header length, DSCP, flags, TTL and the loopback source and destination addresses.*

```
    [Stream index: 0]
Transmission Control Protocol, Src Port: 80, Dst Port: 57338, Seq: 1, Ack: 84, Len: 352
    Source Port: 80
    Destination Port: 57338
    [Stream index: 0]
    [Stream Packet Number: 6]
    [Conversation completeness: Incomplete, DATA (15)]
        ..0. .... = RST: Absent
        ...0 .... = FIN: Absent
        .... 1... = Data: Present
        .... .1.. = ACK: Present
        .... ..1. = SYN-ACK: Present
        .... ...1 = SYN: Present
        [Completeness Flags: ··DASS]
    [TCP Segment Len: 352]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 335975065
    [Next Sequence Number: 353    (relative sequence number)]
    Acknowledgment Number: 84    (relative ack number)
    Acknowledgment number (raw): 2637494999
    1000 .... = Header Length: 32 bytes (8)
    Flags: 0×018 (PSH, ACK)
        000. .... .... = Reserved: Not set
        ...0 .... .... = Accurate ECN: Not set
```

Figure 36: *TCP layer of frame 6, showing ports 80 and 57338, stream index, conversation completeness flags and a 352-byte segment.*

```
        .... 0... .... = Congestion Window Reduced: Not set
        .... .0.. .... = ECN-Echo: Not set
        .... ..0. .... = Urgent: Not set
        .... ...1 .... = Acknowledgment: Set
        .... .... 1... = Push: Set
        .... .... .0.. = Reset: Not set
        .... .... ..0. = Syn: Not set
        .... .... ...0 = Fin: Not set
        [TCP Flags: ·····AP···]
    Window: 512
    [Calculated window size: 65536]
    [Window size scaling factor: 128]
    Checksum: 0×ff88 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
    Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
        TCP Option - No-Operation (NOP)
            Kind: No-Operation (1)
        TCP Option - No-Operation (NOP)
            Kind: No-Operation (1)
        TCP Option - Timestamps: TSval 4102401192, TSecr 4102401191
            Kind: Time Stamp Option (8)
            Length: 10
            Timestamp value: 4102401192
```

Figure 37: *TCP flag bits and options for frame 6, including the PSH/ACK combination, window scaling and the timestamp option.*

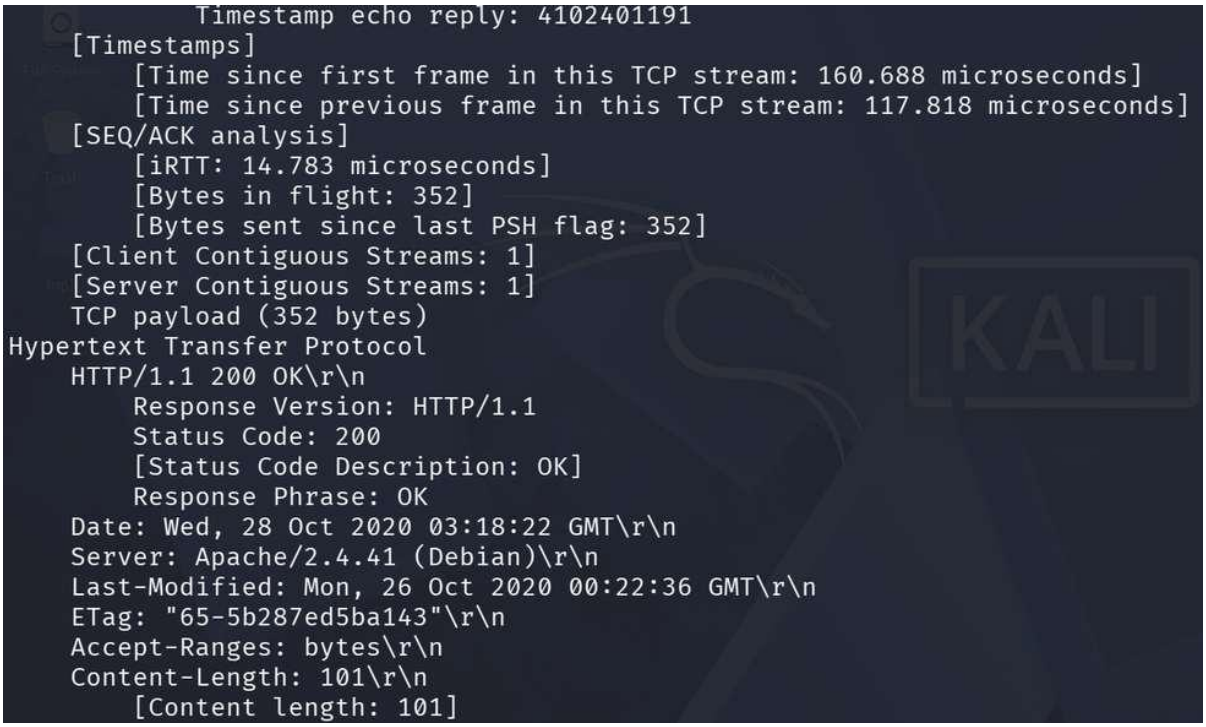


Figure 38: *HTTP response headers within frame 6: status 200 OK, Apache/2.4.41 (Debian), ETag and a content length of 101 bytes.*

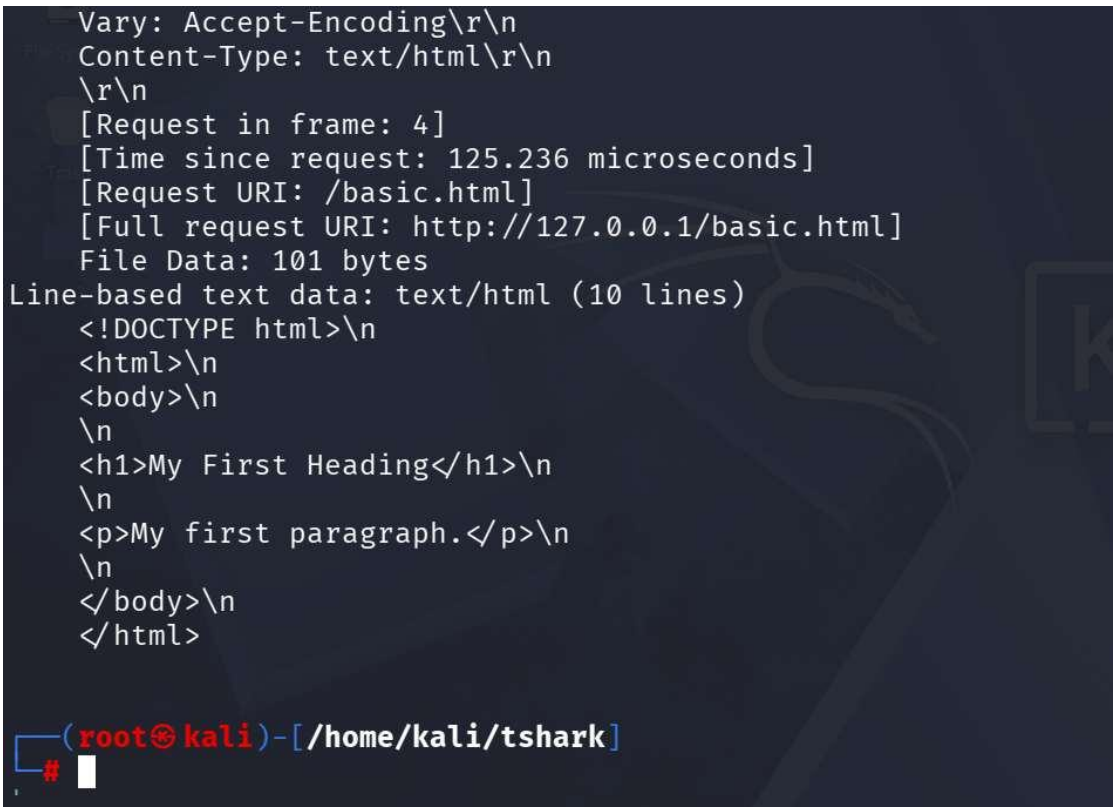


Figure 39: *The decoded HTTP response body, recovering the ten-line HTML document served at http://127.0.0.1/basic.html.*

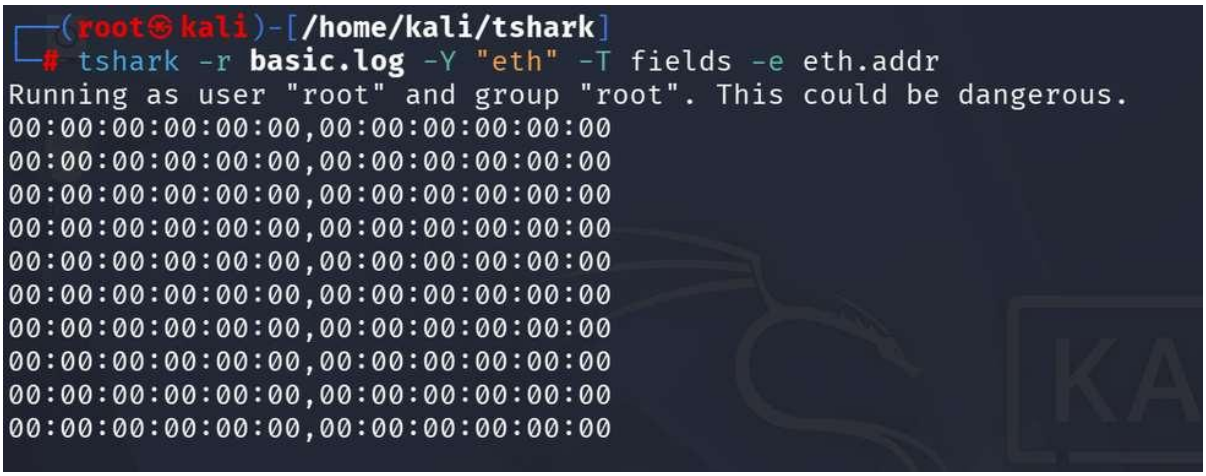


Figure 40: *Extraction of eth.addr across all frames; the all-zero addresses confirm the traffic was captured on the loopback interface.*

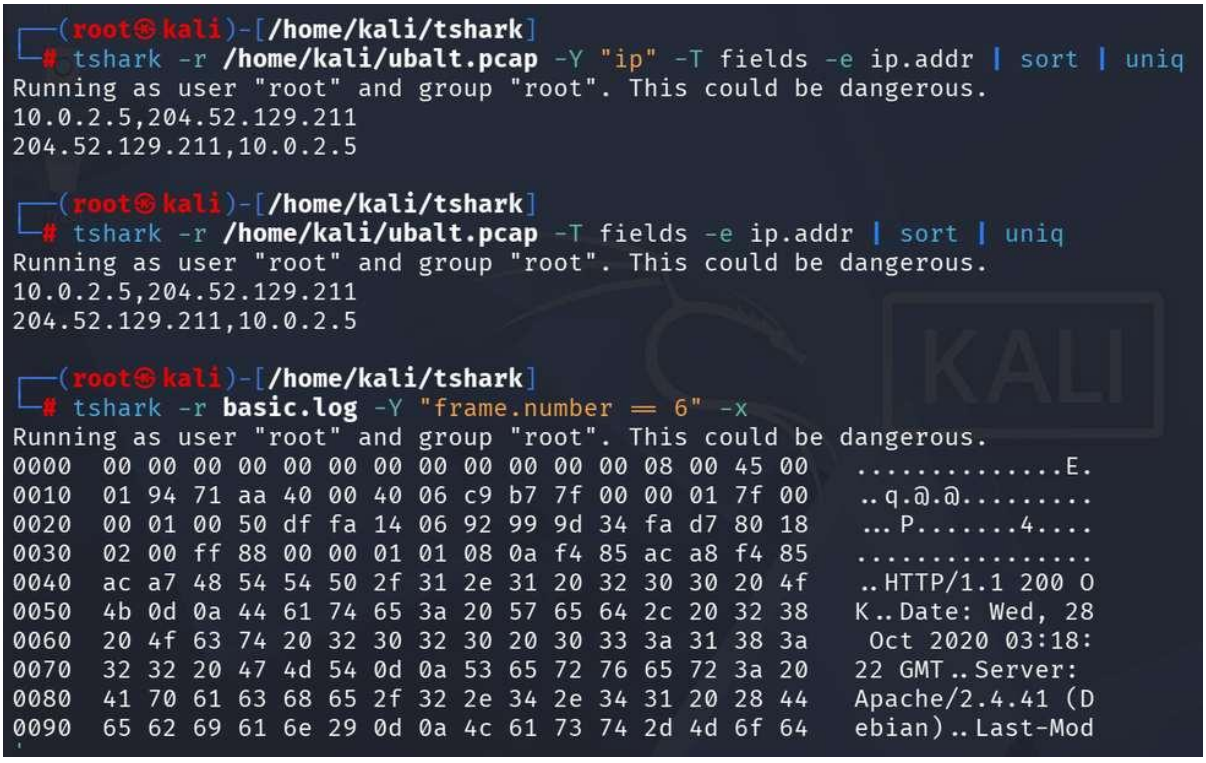


Figure 41: *Extraction and deduplication of ip.addr from ubalt.pcap, and a hex-and-ASCII dump (-x) of frame 6 from basic.log.*

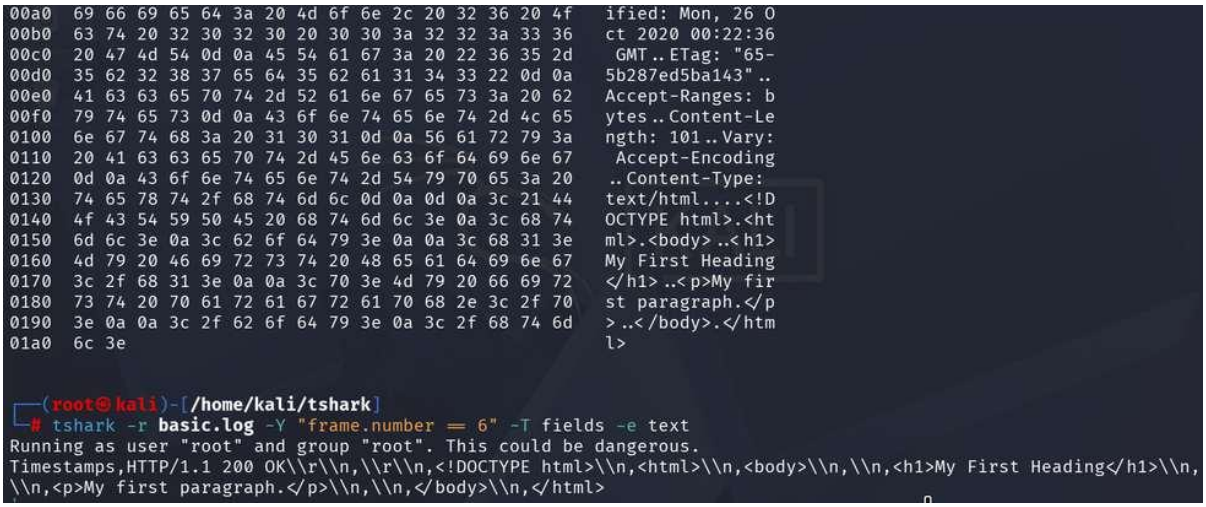


Figure 42: *Continuation of the hex dump showing the HTTP headers and HTML body in ASCII, followed by extraction of the reassembled text field with -T fields -e text.*

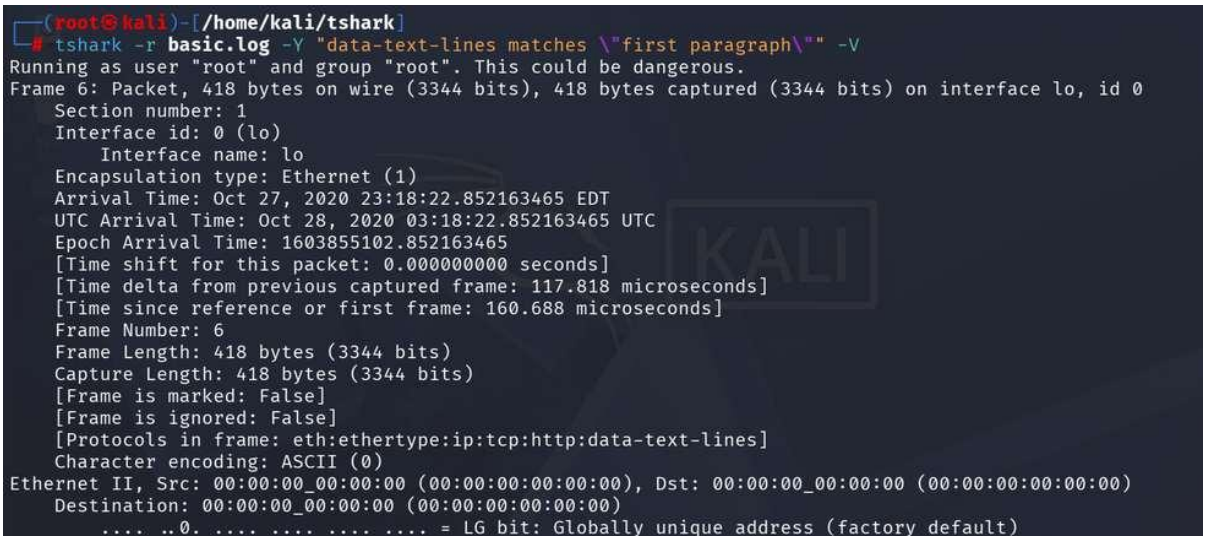


Figure 43: *Verbose output for the frame selected by the regular-expression filter data-text-lines matches "first paragraph".*

```
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Source: 00:00:00_00:00:00 (00:00:00:00:00:00)
        .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0×0800)
    [Stream index: 0]
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0×00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 404
    Identification: 0×71aa (29098)
    010. .... = Flags: 0×2, Don't fragment
        0... .... = Reserved bit: Not set
        .1.. .... = Don't fragment: Set
        ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0×c9b7 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 127.0.0.1
```

Figure 44: *Ethernet and IPv4 detail from the same verbose output, used to confirm the filter selected the intended frame.*

Regular expressions and text processing

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "data-text-lines matches \"first paragraph\"" -V | grep -E "first paragraph" --color
Running as user "root" and group "root". This could be dangerous.
    <p>My first paragraph.</p>\n

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "frame.number == 6" -V | grep -E "<.+?>"
Running as user "root" and group "root". This could be dangerous.
    <!DOCTYPE html>\n
    <html>\n
    <body>\n
    <h1>My First Heading</h1>\n
    <p>My first paragraph.</p>\n
    </body>\n
    </html>

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "http.user_agent" -V | grep -Ei "agent"
Running as user "root" and group "root". This could be dangerous.
    User-Agent: curl/7.67.0\r\n

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 45: *Combining tshark -V with grep -E and --color to highlight a literal phrase, an HTML tag pattern, and the User-Agent header value curl/7.67.0.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -Y "http.request" -T fields -e http.host -e http.user_agent
Running as user "root" and group "root". This could be dangerous.
127.0.0.1       curl/7.67.0

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -V | grep ".html" --color
Running as user "root" and group "root". This could be dangerous.
    GET /basic.html HTTP/1.1\r\n
        Request URI: /basic.html
    [Full request URI: http://127.0.0.1/basic.html]
    Content-Type: text/html\r\n
    [Request URI: /basic.html]
    [Full request URI: http://127.0.0.1/basic.html]
Line-based text data: text/html (10 lines)
    <!DOCTYPE html>\n
    <html>\n
    </html>

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -V | grep -E "([0-9]{1,3}\.){3}[0-9]{1,3}" --color
Running as user "root" and group "root". This could be dangerous.
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
```

Figure 46: *Extraction of http.host and http.user_agent, keyword highlighting of ".html", and a dotted-quad regular expression highlighting IPv4 addresses.*

```
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
    Host: 127.0.0.1\r\n
    [Full request URI: http://127.0.0.1/basic.html]
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
    [Full request URI: http://127.0.0.1/basic.html]
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
```

Figure 47: *Continuation of the IPv4 regular-expression output, with every source and destination address matched in the verbose dissection.*

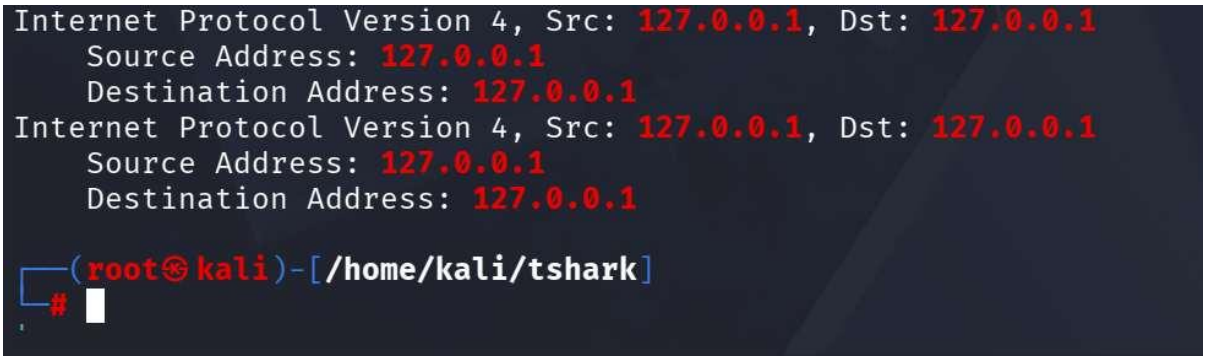


Figure 48: *Final lines of the IPv4 regular-expression output.*

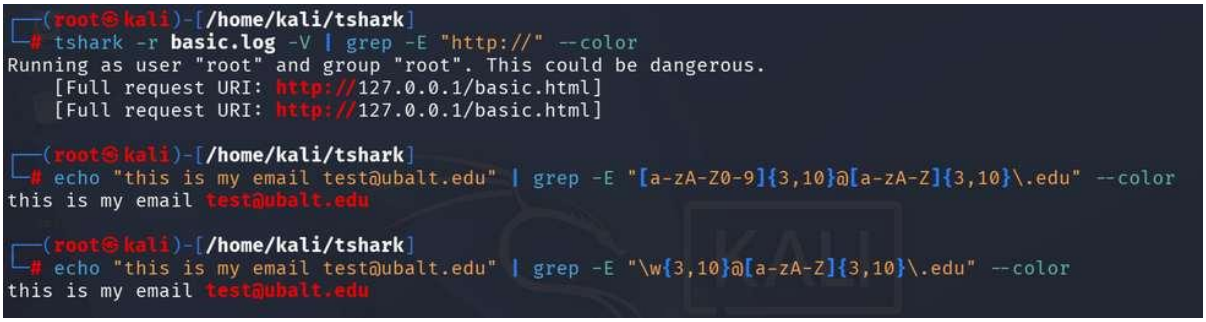


Figure 49: *Highlighting of URL prefixes in the capture, followed by two email-address regular expressions tested against a sample string containing test@ubalt.edu.*

Stream reassembly

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r basic.log -z follow,tcp,ascii,0 -Y http
Running as user "root" and group "root". This could be dangerous.
    4 0.000035452    127.0.0.1 → 127.0.0.1    HTTP 149 GET /basic.html HTTP/1.1
    6 0.000160688    127.0.0.1 → 127.0.0.1    HTTP 418 HTTP/1.1 200 OK  (text/html)

===================================================================
Follow: tcp,ascii
Filter: tcp.stream eq 0
Node 0: 127.0.0.1:57338
Node 1: 127.0.0.1:80
83
GET /basic.html HTTP/1.1
Host: 127.0.0.1
User-Agent: curl/7.67.0
Accept: */*


        352
HTTP/1.1 200 OK
Date: Wed, 28 Oct 2020 03:18:22 GMT
Server: Apache/2.4.41 (Debian)
Last-Modified: Mon, 26 Oct 2020 00:22:36 GMT
ETag: "65-5b287ed5ba143"
Accept-Ranges: bytes
```

Figure 50: *TCP stream reassembly with -z follow,tcp,ascii,0, presenting the client request and server response as a single readable conversation.*

```
Content-Length: 101
Vary: Accept-Encoding
Content-Type: text/html

<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>
</html>
===================================================================

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 51: *Continuation of the followed stream, showing the complete HTML document returned by the server.*

Field and protocol dictionaries

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -G protocols | grep http --color
Running as user "root" and group "root". This could be dangerous.
Binary representation of HTTP Messages  Binary HTTP     bhttp   T       T       T
Hypertext Transfer Protocol     HTTP    http    T       T       T
HyperText Transfer Protocol 2   HTTP2   http2   T       T       T
Hypertext Transfer Protocol Version 3   HTTP3   http3   T       T       T
Media Type: message/http        message/http    message-http    T       T       T

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -G fields | grep -E "http\." --color
Running as user "root" and group "root". This could be dangerous.
F       Frame   bhttp.frame     FT_UINT64       bhttp   BASE_HEX        0x0
F       Framing indicator       bhttp.framing_indicator FT_UINT64       bhttp   BASE_HEX        0x0
F       Request control data    bhttp.request   FT_NONE bhttp           0x0
F       Request method length   bhttp.request.method_len        FT_UINT64       bhttp   BASE_DEC        0x0
F       Request method  bhttp.request.method    FT_STRING       bhttp           0x0
F       Request scheme length   bhttp.request.scheme_len        FT_UINT64       bhttp   BASE_DEC        0x0
F       Request scheme  bhttp.request.scheme    FT_STRING       bhttp           0x0
F       Request authority length        bhttp.request.authority_len     FT_UINT64       bhttp   BASE_DEC        0
x0
F       Request authority       bhttp.request.authority FT_STRING       bhttp           0x0
F       Request path length     bhttp.request.path_len  FT_UINT64       bhttp   BASE_DEC        0x0
F       Request path    bhttp.request.path      FT_STRING       bhttp           0x0
F       Known-length informational response control data        bhttp.response.informational    FT_NONE bhttp  0
```

Figure 52: *Enumeration of HTTP protocol entries with tshark -G protocols and of HTTP field names with tshark -G fields, used to identify valid filter fields.*

```
x0
F       Final response control data     bhttp.response.final    FT_NONE bhttp           0x0
F       Status code     bhttp.response.final.status     FT_UINT64       bhttp   BASE_DEC        0x0
F       Status code     bhttp.response.informational.status     FT_UINT64       bhttp   BASE_DEC        0x0
F       Known-length field section      bhttp.known_length_field_section        FT_NONE bhttp           0x0
F       Known-length field section length       bhttp.known_length_field_section.len    FT_UINT64       bhttp  B
ASE_DEC 0x0
F       Indeterminate-length field section      bhttp.indeterminate_length_field_section        FT_NONE bhttp  0
x0
F       Indeterminate-length content terminator bhttp.indeterminate_content_terminator  FT_UINT64       bhttp  B
ASE_DEC 0x0
F       Name length     bhttp.name_len  FT_UINT64       bhttp   BASE_DEC        0x0
F       Name    bhttp.name      FT_STRING       bhttp           0x0
F       Value length    bhttp.value_len FT_UINT64       bhttp   BASE_DEC        0x0
F       Value   bhttp.value     FT_STRING       bhttp           0x0
F       Known-length content    bhttp.known_length_content      FT_NONE bhttp           0x0
F       Content length  bhttp.known_length_content.len  FT_UINT64       bhttp   BASE_DEC        0x0
F       Content bhttp.known_length_content.content      FT_BYTES        bhttp           0x0
F       Indeterminate-length content    bhttp.indeterminate_length_content      FT_NONE bhttp           0x0
F       Chunk length    bhttp.indeterminate_length_content.chunk_len    FT_UINT64       bhttp   BASE_DEC        0
x0
F       Padding Length  bhttp.padding_length    FT_UINT32       bhttp   BASE_DEC        0x0
F       Notification    http.notification       FT_BOOLEAN      http    0       0x0     true if HTTP notificatio
```

Figure 53: *Continuation of the HTTP field listing, showing binary-HTTP content and chunk-length fields.*



Figure 54: *HTTP request and response field names, including http.request.method, http.request.uri, http.request.full_uri and http.response.code.*

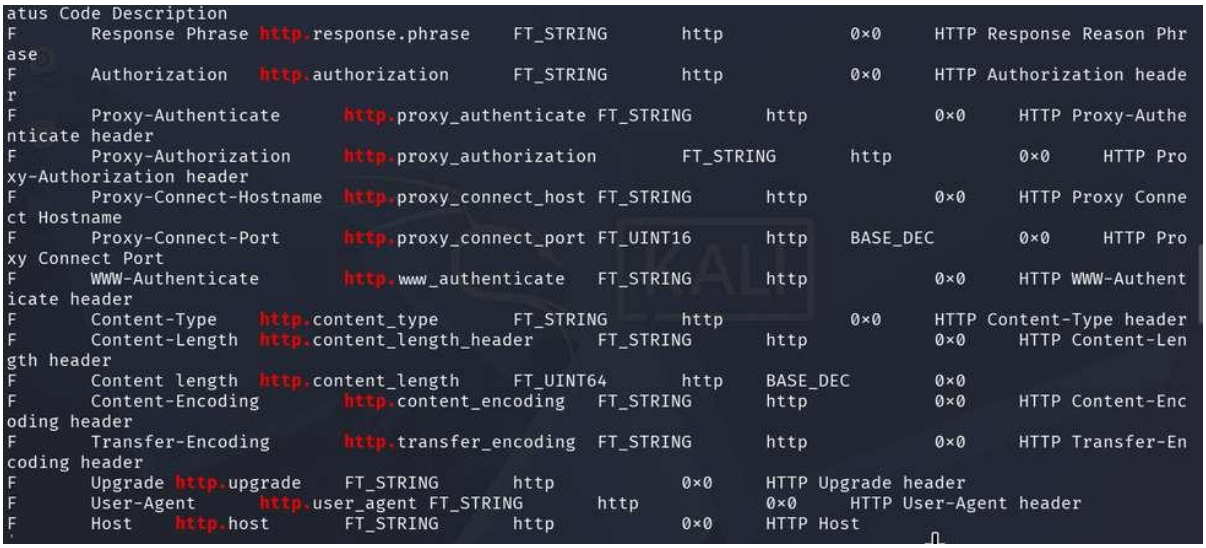


Figure 55: *HTTP header field names, including http.content_type, http.user_agent and http.host.*

```
F       Content-Range   http.content_range      FT_STRING       http            0x0     HTTP Content-Range
F       Connection      http.connection FT_STRING       http            0x0     HTTP Connection
F       Cookie  http.cookie     FT_STRING       http            0x0     HTTP Cookie
F       Cookie pair     http.cookie_pair        FT_STRING       http            0x0     A name/value HTTP cookie
 pair
F       Accept  http.accept     FT_STRING       http            0x0     HTTP Accept
F       Referer http.referer    FT_STRING       http            0x0     HTTP Referer
F       Accept-Language http.accept_language    FT_STRING       http            0x0     HTTP Accept Language
F       Accept Encoding http.accept_encoding    FT_STRING       http            0x0     HTTP Accept Encoding
F       Date    http.date       FT_STRING       http            0x0     HTTP Date
F       Cache-Control   http.cache_control      FT_STRING       http            0x0     HTTP Cache Control
F       Server  http.server     FT_STRING       http            0x0     HTTP Server
F       Location        http.location   FT_STRING       http            0x0     HTTP Location
F       Sec-WebSocket-Accept    http.sec_websocket_accept       FT_STRING       http            0x0
F       Sec-WebSocket-Extensions        http.sec_websocket_extensions   FT_STRING       http            0x0
F       Sec-WebSocket-Key       http.sec_websocket_key  FT_STRING       http            0x0
F       Sec-WebSocket-Protocol  http.sec_websocket_protocol     FT_STRING       http            0x0
F       Sec-WebSocket-Version   http.sec_websocket_version      FT_STRING       http            0x0
F       Set-Cookie      http.set_cookie FT_STRING       http            0x0     HTTP Set Cookie
F       Last-Modified   http.last_modified      FT_STRING       http            0x0     HTTP Last Modified
F       X-Forwarded-For http.x_forwarded_for    FT_STRING       http            0x0     HTTP X-Forwarded-For
F       HTTP2-Settings  http.http2_settings     FT_STRING       http            0x0
F       Request in frame        http.request_in FT_FRAMENUM     http            0x0     This packet is a respons
e to the packet with this number
F       Response in frame       http.response_in        FT_FRAMENUM     http            0x0     This packet will
 be responded in the packet with this number
```

Figure 56: *Further HTTP field definitions returned by the -G fields dictionary.*

```
equest was sent
F       trailer-part    http.chunked_trailer_part       FT_STRING       http            0x0     Optional trailer
 in a chunked body
F       Chunk boundary  http.chunk_boundary     FT_BYTES        http            0x0
F       Chunk size      http.chunk_size FT_UINT32       http    BASE_DEC        0x0
F       Chunk data      http.chunk_data FT_BYTES        http            0x0
F       File Data       http.file_data  FT_BYTES        http            0x0
F       Unknown header  http.unknown_header     FT_STRING       http            0x0
F       HTTP2 Settings URI      http.http2_settings_uri FT_BYTES        http            0x0
F       Reassembled HTTP Chunked Body fragments http.body.fragments     FT_NONE http            0x0
F       HTTP Chunked Body fragment      http.body.fragment      FT_FRAMENUM     http            0x0
F       HTTP Chunked Body fragment overlap      http.body.fragment.overlap      FT_BOOLEAN      http    0       0
x0
F       HTTP Chunked Body fragment overlapping with conflicting data    http.body.fragment.overlap.conflicts    F
T_BOOLEAN       http    0       0x0
F       HTTP Chunked Body has multiple tail fragments   http.body.fragment.multiple_tails       FT_BOOLEAN      h
ttp     0       0x0
F       HTTP Chunked Body fragment too long     http.body.fragment.too_long_fragment    FT_BOOLEAN      http    0
0x0
F       HTTP Chunked Body defragment error      http.body.fragment.error        FT_FRAMENUM     http            0
x0
F       HTTP Chunked Body fragment count        http.body.fragment.count        FT_UINT32       http    BASE_DEC
0x0
F       Reassembled in  http.body.reassembled.in        FT_FRAMENUM     http            0x0
F       Reassembled length      http.body.reassembled.length    FT_UINT32       http    BASE_DEC        0x0
F       Reassembled data        http.body.reassembled.data      FT_BYTES        http            0x0
```

Figure 57: *Further HTTP field definitions returned by the -G fields dictionary.*

```
F       HTTP Chunked Body segment       http.body.segment       FT_BYTES        http            0x0
F       The Content-Length and Transfer-Encoding header must not be set together        http.te_and_length      F
T_NONE  http            0x0
F       Unknown transfer coding name in Transfer-Encoding header        http.te_unknown FT_NONE http            0
x0
F       HTTP body subdissector failed, trying heuristic subdissector    http.subdissector_failed        FT_NONEh
ttp             0x0
F       Unencrypted HTTP protocol detected over encrypted port, could indicate a dangerous misconfiguration.    h
ttp.tls_port    FT_NONE http            0x0
F       Excess data after a body (not a new request/response), previous Content-Length bogus?   http.excess_data
FT_NONE http            0x0
F       Leading CRLF previous message in the stream may have extra CRLF http.leading_crlf       FT_NONE http    0
x0
F       Illegal characters found in header name http.bad_header_name    FT_NONE http            0x0
F       Decompression failed    http.decompression_failed       FT_NONE http            0x0
F       Decompression disabled  http.decompression_disabled     FT_NONE http            0x0

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 58: *HTTP expert-information fields, including malformed-header and decompression-failure indicators.*



Figure 59: *Enumeration of TCP-related protocols using a word-boundary regular expression against tshark -G protocols.*

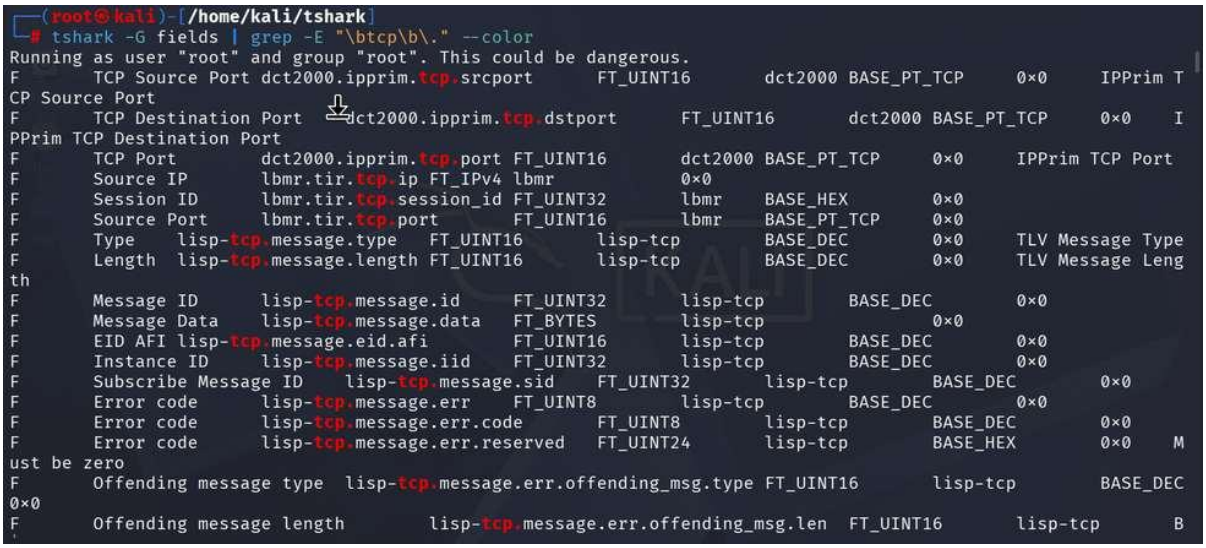


Figure 60: *TCP field names returned by a word-boundary regular expression against tshark -G fields.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -G protocols | grep frame --color
Running as user "root" and group "root". This could be dangerous.
Apple Wireless Direct Link action frame AWDL    awdl    T       T       T
Apple Wireless Direct Link data frame   AWDL data       awdl_data       T       T       T
Classical IP frame      CLIP    clip    T       T       T
Compound Frames (Deprecated)    Compound Frames (Deprecated)    lcp.opt.compound_frames T       T       T
EtherCAT frame header   ETHERCAT        ecatf   T       T       T
Frame   Frame   frame   T       T       F
IEEE 802 Tagged Frame   IEEE 802 Tagged Frame   bcp_ncp.opt.ieee_802_tagged_frame       T       T       T
IEEE 802.11 wireless LAN aggregate frame        IEEE 802.11 Aggregate Data      wlan_aggregate  T       T       T
IEEE 802.11 wireless LAN extension frame        IEEE 802.11 EXT wlan_ext        T       T       T
mac-lte-framed  MAC-LTE-FRAMED  mac-lte-framed  T       T       T
mac-nr-framed   MAC-NR-FRAMED   mac-nr-framed   T       T       T
Short Frame     Short frame     _ws.short       T       T       F

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -G fields | grep -e "frame\.number" --color
Running as user "root" and group "root". This could be dangerous.
F       Number of Periods       bthci_cmd.external_frame.number_of_periods      FT_UINT8        bthci_cmd       B
ASE_DEC 0x0
F       Frame number    dvb-s2_table.frame.number       FT_UINT8        dvb-s2_table    BASE_DEC        0x1f
F       Frame number    dvb-s2_table.desc.cmed.frame.number     FT_UINT8        dvb-s2_table    BASE_DEC        0
x0
F       Frame Number    frame.number    FT_UINT32       frame   BASE_DEC        0x0
```

Figure 61: *Continuation of the TCP field enumeration.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -G fields | grep -e "frame\.time" --color
Running as user "root" and group "root". This could be dangerous.
F       Timestamp Microseconds  btsnoop.frame.timestamp_microseconds    FT_ABSOLUTE_TIME        btsnoop         0
x0
F       Timestamp       dpnet.cframe.timestamp  FT_UINT32       dpnet   BASE_DEC        0x0
F       Arrival Time    frame.time      FT_ABSOLUTE_TIME        frame           0x0     Absolute time when this
frame was captured, in local time
F       UTC Arrival Time        frame.time_utc  FT_ABSOLUTE_TIME        frame           0x0     Absolute time wh
en this frame was captured, in Coordinated Universal Time (UTC)
F       Epoch Arrival Time      frame.time_epoch        FT_ABSOLUTE_TIME        frame           0x0     Absolute
 time when this frame was captured, in Epoch time (also known as Unix time)
F       Time delta from previous captured frame frame.time_delta        FT_RELATIVE_TIME        frame           0
x0
F       Time delta from previous displayed frame        frame.time_delta_displayed      FT_RELATIVE_TIME        f
rame            0x0
F       Time since reference or first frame     frame.time_relative     FT_RELATIVE_TIME        frame           0
x0      Time relative to time reference or first frame
F       Time since start of capturing   frame.time_relative_capture_start       FT_RELATIVE_TIME        frame   0
x0      Time relative to the capture start
F       Arrival Time: Fractional second out of range (0-999999999)      frame.time_invalid      FT_NONE frame   0
x0
F       Timestamp       rdp_egfx.startframe.timestamp   FT_UINT32       rdp_egfx        BASE_DEC        0x0
F       Timestamp       tdma-v1.msg.start_of_frame.timestamp    FT_UINT64       rtmac   BASE_DEC        0x0     T
DMA Timestamp
```

Figure 62: *Continuation of the TCP field enumeration.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -G fields | grep -P "^(?=.*protocol)(?=.*name)"
Running as user "root" and group "root". This could be dangerous.
F       Protocol        amqp.init.protocol      FT_STRING       amqp            0x0     Protocol name
F       Protocol Name Length    bittorrent.protocol.name.length FT_UINT8        bittorrent      BASE_DEC        0
x0
F       Protocol Name   bittorrent.protocol.name        FT_STRING       bittorrent              0x0
F       DCT2000 protocol        dct2000.protocol        FT_STRING       dct2000         0x0     Original (DCT200
0) protocol name
F       Protocol Name   kafka.protocol_name     FT_STRING       kafka           0x0
F       3GPP protocol   log3gpp.protocol        FT_STRING       log3gpp         0x0     Original 3GPP protocol n
ame
F       Protocol Vendor ID      matter.payload.protocol_vendor_id       FT_UINT16       matter  BASE_HEX        0
x0      Vendor ID namespace for the protocol ID
F       Protocol Name   ncp.protocol_name       FT_UINT_STRING  ncp             0x0
F       Layer 7 protocol name   cflow.pie.ntop.l7_proto_name    FT_STRING       cflow           0x0
F       DB Protocol Name        cflow.pie.netscaler.db-protocol-name    FT_UINT8        cflow   BASE_DEC        0
x0      Database protocol
F       Unknown nameservice protocol version    olsr.ns.version.unknown FT_NONE olsr            0x0
F       Protocol        snort.protocol  FT_STRINGZ      snort           0x0     Protocol name as given in the ru
le
F       authorization-protocol-name-length      x11.authorization-protocol-name-length  FT_UINT16       x11     B
ASE_DEC 0x0
F       authorization-protocol-name     x11.authorization-protocol-name FT_STRING       x11             0x0

┌──(root㉿kali)-[/home/kali/tshark]
```

Figure 63: *A PCRE lookahead expression (grep -P) selecting only field definitions that contain both "protocol" and "name".*

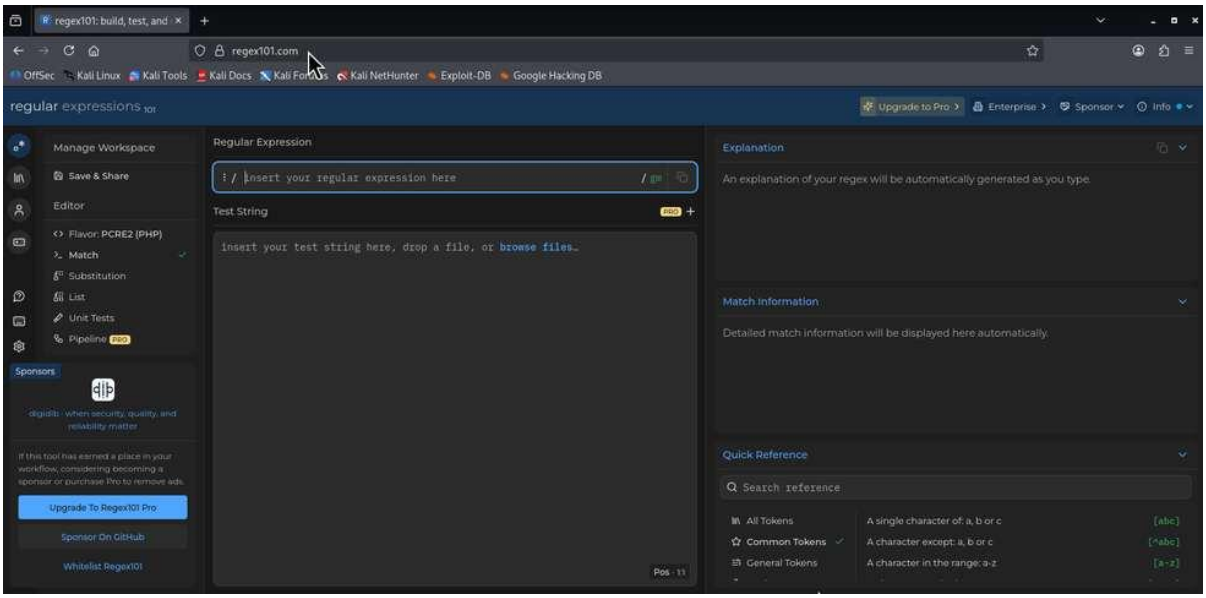


Figure 64: *The regex101.com workspace used to construct and validate the regular expressions applied in this lab.*

End-to-end capture exercise

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -f "host 204.52.129.211" -i eth0 -w ubalt.pcap -c 500
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
500
```

Figure 65: *Live capture of 500 packets to ubalt.pcap using the capture filter "host 204.52.129.211".*

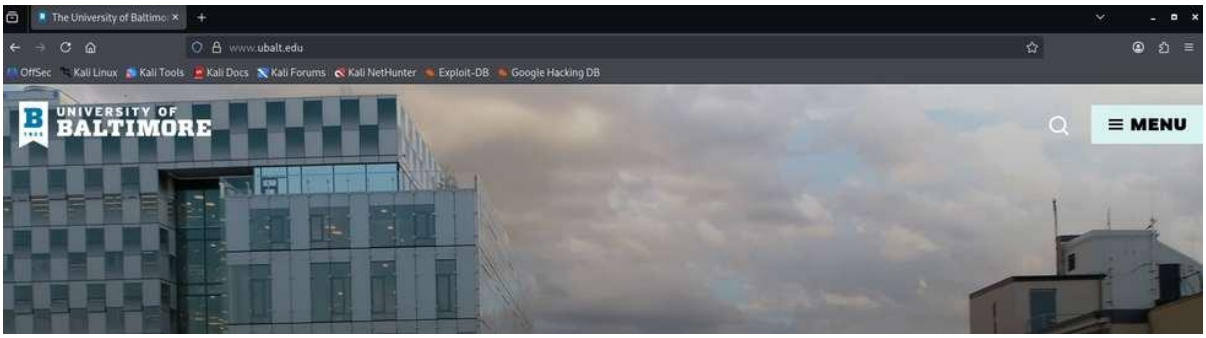


Figure 66: *The University of Baltimore website loaded in Firefox, generating the HTTP traffic captured in Figure 65.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r /home/kali/ubalt.pcap -Y "frame.number == 14 && data-text-lines" -V | grep -E "[0-9]{3}\.[0-9]{3}\
.[0-9]{4}" --color
Running as user "root" and group "root". This could be dangerous.
    <p>General Info:<br /><span><a style="color: white;" href="tel:+14108374200">410.837.4200</a> </span></p>\n

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r /home/kali/ubalt.pcap -Y "frame.number == 14 && data-text-lines" -V | grep -E "([0-9A-Fa-f]{2}[:-]
){5}([0-9A-Fa-f]{2})" --color
Running as user "root" and group "root". This could be dangerous.
Ethernet II, Src: 52:54:00:12:35:00 (52:54:00:12:35:00), Dst: PCSSystemtec_4e:f7:72 (08:00:27:4e:f7:72)
    Destination: PCSSystemtec_4e:f7:72 (08:00:27:4e:f7:72)
    Source: 52:54:00:12:35:00 (52:54:00:12:35:00)

┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r /home/kali/ubalt.pcap -Y "eth" -T fields -e eth.addr | sort | uniq
Running as user "root" and group "root". This could be dangerous.
08:00:27:4e:f7:72,52:54:00:12:35:00
52:54:00:12:35:00,08:00:27:4e:f7:72
```

Figure 67: *Regular-expression extraction from ubalt.pcap: a telephone number recovered from the HTML payload of frame 14, a MAC-address pattern matching both Ethernet addresses, and the deduplicated eth.addr list.*

```
┌──(root㉿kali)-[/home/kali/tshark]
└─# tshark -r /home/kali/ubalt.pcap -T fields -e ip.addr | sort | uniq
Running as user "root" and group "root". This could be dangerous.
10.0.2.5,204.52.129.211
204.52.129.211,10.0.2.5

┌──(root㉿kali)-[/home/kali/tshark]
└─#
```

Figure 68: *Deduplicated IP addresses in ubalt.pcap, confirming a single conversation between the capture host 10.0.2.5 and the web server 204.52.129.211.*

5.3 Command-Line Analysis of the Evidence (Figures 69–81)

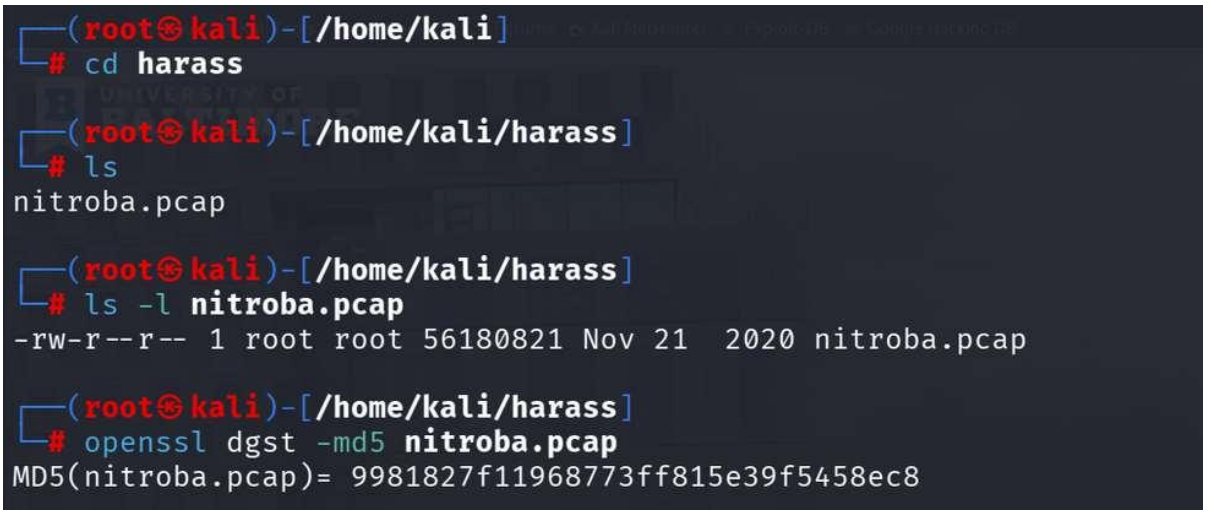


Figure 69: *Return to the Nitroba evidence directory and re-verification of the MD5 hash, which matches the acquisition hash recorded in Figure 2.*

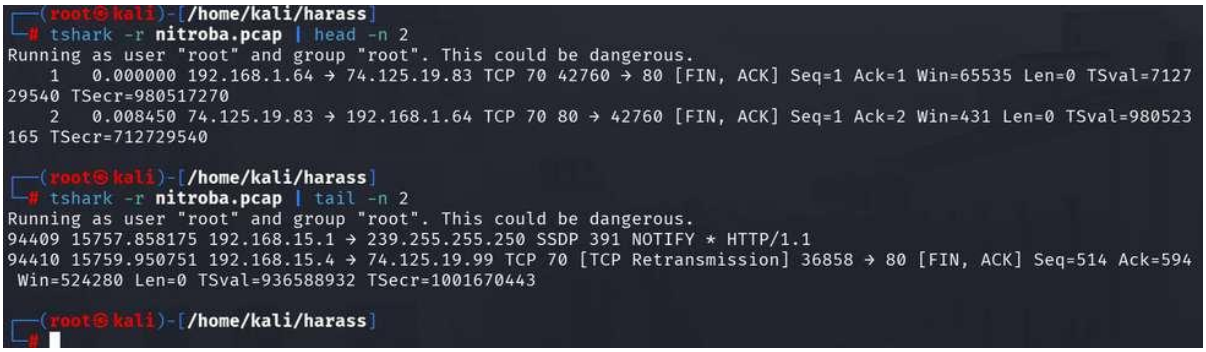


Figure 70: *First and last frames of nitroba.pcap, establishing the temporal boundaries of the capture at 94,410 packets.*

```
┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -qz io,stat,0,"COUNT(frame) frame"
Running as user "root" and group "root". This could be dangerous.

===================================
| IO Statistics                   |
|                                 |
| Duration: 15760.0 secs          |
| Interval: 15760.0 secs          |
|                                 |
| Col  1: COUNT(frame) frame      |
|---------------------------------|
|                    |1     |     |
| Interval           | COUNT |     |
|--------------------------|     |
|     0.0 <> 15760.0 | 94410 |     |
===================================
```

Figure 71: *IO statistics for the full capture: 94,410 frames spanning a duration of 15,760 seconds (approximately 4 hours 23 minutes).*

```
┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -T fields -e frame.time -e frame | head -n 3
Running as user "root" and group "root". This could be dangerous.
2008-07-21T21:51:07.095278000-0400      Frame 1: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bit
s)
2008-07-21T21:51:07.103728000-0400      Frame 2: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bit
s)
2008-07-21T21:51:07.114897000-0400      Frame 3: Packet, 1421 bytes on wire (11368 bits), 1421 bytes captured (1
1368 bits)

┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "http.request.method == \"GET\" && http.host==\"www.willselfdestruct.com\" "
Running as user "root" and group "root". This could be dangerous.
82936 15156.730593 192.168.15.4 → 69.25.94.22  HTTP 596 GET /secure/submit HTTP/1.1
82985 15157.030671 192.168.15.4 → 69.25.94.22  HTTP 355 GET /images/spacer.gif HTTP/1.1
83025 15157.238976 192.168.15.4 → 69.25.94.22  HTTP 356 GET /images/sm-logo.gif HTTP/1.1
83037 15157.279305 192.168.15.4 → 69.25.94.22  HTTP 361 GET /images/warning-home.gif HTTP/1.1
83072 15157.435074 192.168.15.4 → 69.25.94.22  HTTP 356 GET /images/body-bk.gif HTTP/1.1
83087 15157.503801 192.168.15.4 → 69.25.94.22  HTTP 358 GET /images/bttn-send.gif HTTP/1.1
83162 15157.756987 192.168.15.4 → 69.25.94.22  HTTP 361 GET /images/bridge_small.gif HTTP/1.1
83614 15197.468887 192.168.15.4 → 69.25.94.22  HTTP 461 GET /secure/success HTTP/1.1
83654 15197.806879 192.168.15.4 → 69.25.94.22  HTTP 360 GET /images/bk-message.gif HTTP/1.1

┌──(root㉿kali)-[/home/kali/harass]
└─#
```

Figure 72: *Absolute timestamps for the first three frames, followed by all GET requests to www.willselfdestruct.com issued by 192.168.15.4.*

```
┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "http.request.method == "GET" && http.host=="www.willselfdestruct.com"" -T fields
-e frame.time -e frame.time_delta_displayed -e ip.src -e ip.dst -e http.request.uri
Running as user "root" and group "root". This could be dangerous.
2008-07-22T02:03:43.825871000-0400              192.168.15.4    69.25.94.22     /secure/submit
2008-07-22T02:03:44.125949000-0400      0.300078000     192.168.15.4    69.25.94.22     /images/spacer.gif
2008-07-22T02:03:44.334254000-0400      0.208305000     192.168.15.4    69.25.94.22     /images/sm-logo.gif
2008-07-22T02:03:44.374583000-0400      0.040329000     192.168.15.4    69.25.94.22     /images/warning-home.gif
2008-07-22T02:03:44.530352000-0400      0.155769000     192.168.15.4    69.25.94.22     /images/body-bk.gif
2008-07-22T02:03:44.599079000-0400      0.068727000     192.168.15.4    69.25.94.22     /images/bttn-send.gif
2008-07-22T02:03:44.852265000-0400      0.253186000     192.168.15.4    69.25.94.22     /images/bridge_small.gif
2008-07-22T02:04:24.564165000-0400      39.711900000    192.168.15.4    69.25.94.22     /secure/success
2008-07-22T02:04:24.902157000-0400      0.337992000     192.168.15.4    69.25.94.22     /images/bk-message.gif

┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "data-text-lines matches \"secure anonymous E-mail\"" 2>/dev/null
82998 15157.115668  69.25.94.22 → 192.168.15.4 HTTP 75 HTTP/1.1 200 OK  (text/html)
83641 15197.756153  69.25.94.22 → 192.168.15.4 HTTP 1189 HTTP/1.1 200 OK  (text/html)

┌──(root㉿kali)-[/home/kali/harass]
└─#
```

Figure 73: *The same GET requests with timestamps, inter-packet deltas and request URIs, showing the progression from /secure/submit to /secure/success, and the two matching server responses.*

```
┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "(ip.src == 192.168.15.4 ) && (ip.dst == 69.25.94.22) && http.request.method == "P
OST""
Running as user "root" and group "root". This could be dangerous.
83601 15197.216422 192.168.15.4 → 69.25.94.22  HTTP 719 POST /secure/submit HTTP/1.1  (application/x-www-form-ur
lencoded)

┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "frame.number == 83601" -V | grep "lilytuckrige@yahoo.com"
Running as user "root" and group "root". This could be dangerous.
    Form item: "to" = "lilytuckrige@yahoo.com"
        Value: lilytuckrige@yahoo.com

┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "(ip.src == 192.168.15.4 ) && (ip.dst == 69.25.94.22) && http.request.method == "P
OST"" -T fields -e frame.time -e text
Running as user "root" and group "root". This could be dangerous.
2008-07-22T02:04:24.311700000-0400      Timestamps,POST /secure/submit HTTP/1.1\\r\\n,\\r\\n,Form item: "to" = "
lilytuckrige@yahoo.com",Form item: "from" = "",Form item: "subject" = "you can't find us",Form item: "message" =
 "and you can't hide from us.\\r\\n\\r\\nStop teaching.\\r\\n\\r\\nStart running. ",Form item: "type" = "0",Form
 item: "ttl" = "30",Form item: "submit.x" = "92",Form item: "submit.y" = "26"

┌──(root㉿kali)-[/home/kali/harass]
└─#
```

Figure 74: *Isolation of the POST in frame 83601, confirmation that the recipient field contains lilytuckrige@yahoo.com, and extraction of the full form payload with its absolute timestamp.*

```
┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "frame.number == 83601" -V | grep -i "form item" --color
Running as user "root" and group "root". This could be dangerous.
    Form item: "to" = "lilytuckrige@yahoo.com"
    Form item: "from" = ""
    Form item: "subject" = "you can't find us"
    Form item: "message" = "and you can't hide from us.\r\n\r\nStop teaching.\r\n\r\nStart running. "
    Form item: "type" = "0"
    Form item: "ttl" = "30"
    Form item: "submit.x" = "92"
    Form item: "submit.y" = "26"

┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -G fields | grep urlencoded-form
Running as user "root" and group "root". This could be dangerous.
P       HTML Form URL Encoded   urlencoded-form
F       Key     urlencoded-form.key     FT_STRINGZ      urlencoded-form         0x0
F       Value   urlencoded-form.value   FT_STRINGZ      urlencoded-form         0x0

┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "frame.number == 83601" -V -T fields -e urlencoded-form.key -e urlencoded-form.val
ue
Running as user "root" and group "root". This could be dangerous.
to,from,subject,message,type,ttl,submit.x,submit.y      lilytuckrige@yahoo.com,,you can't find us,and you can't
hide from us.\r\n\r\nStop teaching.\r\n\r\nStart running. ,0,30,92,26
```

Figure 75: *Complete form-item listing for frame 83601, the urlencoded-form field definitions, and key/value extraction of the submitted message.*

```
┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "frame.number == 83601" -T fields -e eth.src
Running as user "root" and group "root". This could be dangerous.
00:17:f2:e2:c0:ce

┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -Y "eth.addr == 00:17:f2:e2:c0:ce" | wc -l
Running as user "root" and group "root". This could be dangerous.
73246
```

Figure 76: *Source MAC address of the POST (00:17:f2:e2:c0:ce) and a frame count showing that this interface accounts for 73,246 frames in the capture.*

```
┌──(root㉿kali)-[/home/kali/harass]
└─# tshark -r nitroba.pcap -qz io,stat,0,"COUNT(eth) eth","eth.addr == 00:17:f2:e2:c0:ce"
Running as user "root" and group "root". This could be dangerous.

================================================
| IO Statistics                                |
|                                              |
| Duration: 15760.0 secs                       |
| Interval: 15760.0 secs                       |
|                                              |
| Col  1: COUNT(eth) eth                       |
|      2: eth.addr == 00:17:f2:e2:c0:ce        |
|----------------------------------------------|
|                    |1      |2                |
| Interval           | COUNT | Frames |   Bytes  |
|----------------------------------------------|
|     0.0 <> 15760.0 | 94410 |  73246 | 44742268 |
================================================

┌──(root㉿kali)-[/home/kali/harass]
└─#
```

Figure 77: *IO statistics filtered on eth.addr == 00:17:f2:e2:c0:ce: 73,246 frames totalling 44,742,268 bytes attributable to the suspect device.*

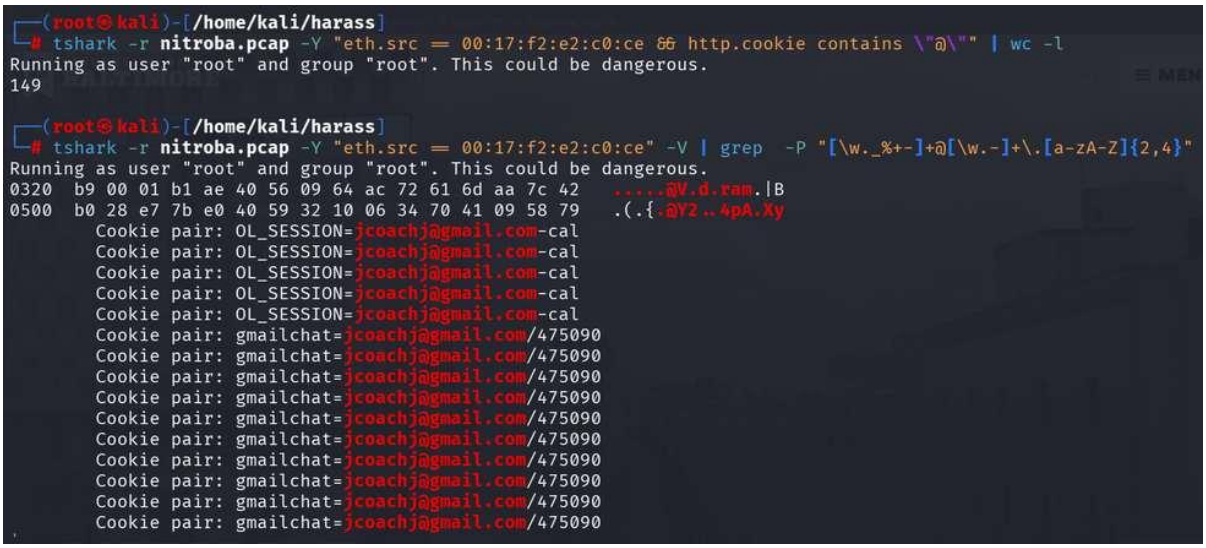


Figure 78: *149 cookie-bearing frames from the suspect MAC, and a regular-expression sweep of the verbose output recovering the OL_SESSION and gmailchat cookies for jcoachj@gmail.com.*



Figure 79: *Continuation of the recovered gmailchat cookie pairs, showing the repeated appearance of the same account identifier across the session.*

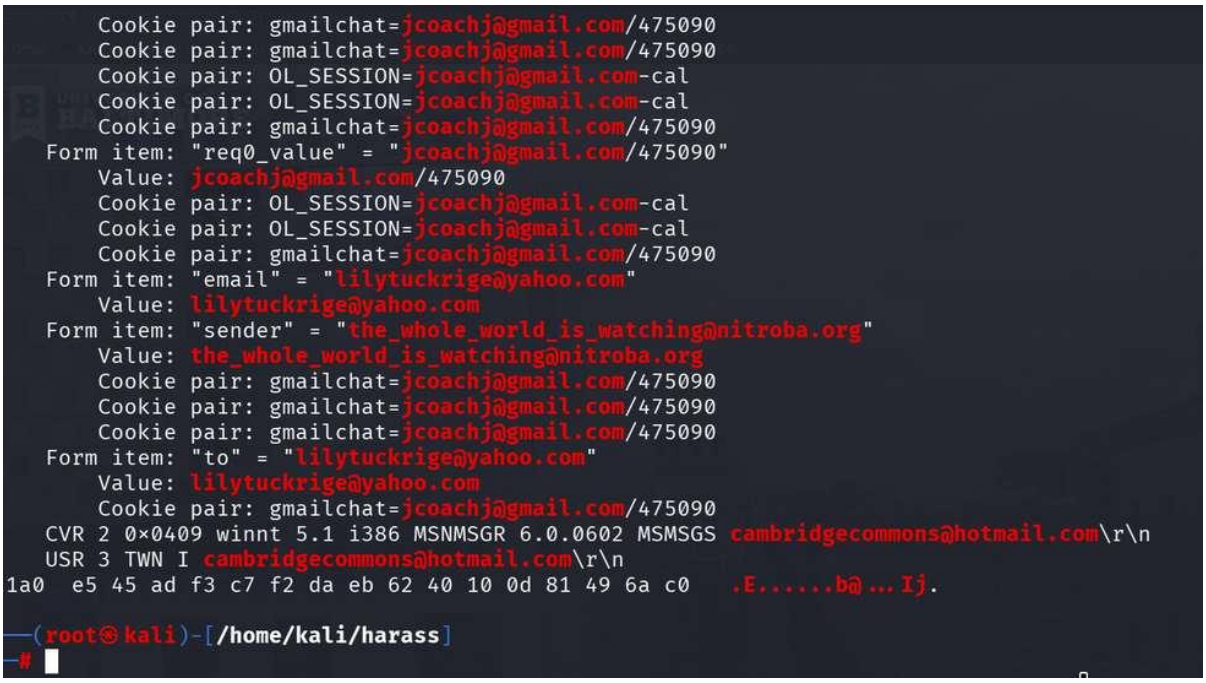


Figure 80: *Recovery of the correlating identifiers: the form fields email = lilytuckrige@yahoo.com and sender = the_whole_world_is_watching@nitroba.org, alongside the Gmail account jcoachj@gmail.com and an MSN Messenger login for cambridgecommons@hotmail.com.*

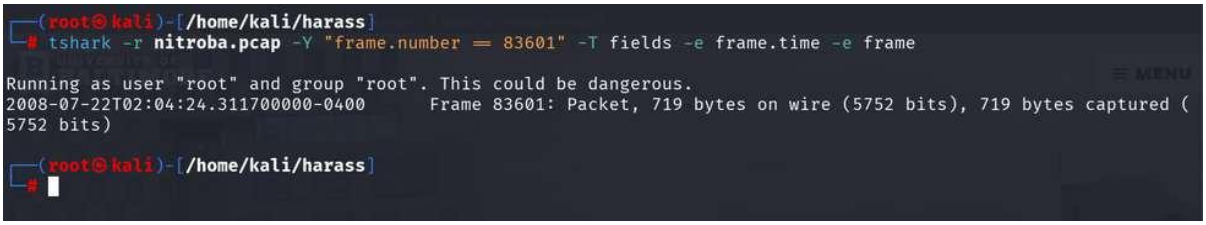


Figure 81: *Absolute timestamp of the harassing POST, frame 83601, recorded at 2008-07-22 02:04:24.311700 -0400.*

6. Analysis and Findings

6.1 The capture and its scope

The evidence file contains 94,410 frames spanning 15,760 seconds, or roughly four hours and twenty-three minutes, beginning at 21:51:07 (-0400) on 21 July 2008 (Figures 70 and 71). The capture is unencrypted HTTP throughout the period of interest, which is what makes the analysis possible: the same traffic carried over TLS today would not yield form contents or cookie values.

6.2 Identification of the transmission mechanism

Filtering for HTTP requests to www.willselfdestruct.com identified a single host, 192.168.15.4, interacting with the service at 69.25.94.22 (Figures 4 and 72). The reassembled HTML of the server response describes the service as a free anonymous, self-destructing email provider (Figure 6). This is a web application that accepts a recipient, a subject and a message body through a form and delivers the message without disclosing the sender, which explains why the complainant could not identify the origin of the messages from the messages themselves.

The request sequence recovered in Figure 73 shows the full interaction in order: a GET to /secure/submit at 02:03:43, a series of interface image requests over the following second, then a GET to /secure/success at 02:04:24, approximately forty seconds later. That forty-second gap is the interval in which the message was composed and submitted.

6.3 Recovery of the message

The submission itself is frame 83601, a 719-byte HTTP POST to /secure/submit carrying an application/x-www-form-urlencoded body, sent at 2008-07-22 02:04:24.311700 (-0400) (Figures 8, 74 and 81). The form fields were recovered in full (Figures 10 and 75):

Form field	Value
to	lilytuckrige@yahoo.com
from	(empty)
subject	you can’t find us

Form field	Value
message	"and you can’t hide from us. / Stop teaching. / Start running."
ttl	30
type	0

The subject line and the message body read as a single continuous sentence, indicating the sender composed them together. The content is threatening and directed at the recipient in her professional capacity. The empty "from" field is consistent with deliberate use of the anonymising feature rather than an incidental omission.

The server accepted the submission: the response to /secure/submit returned HTTP 200 with a 20,126-byte body, and the subsequent /secure/success page returned HTTP 200 with a 15,830-byte body (Figures 13, 14 and 73). The message was therefore delivered, not merely attempted.

6.4 Attribution to a device

The POST originated from 192.168.15.4. An IP address on a shared dormitory network is weak evidence on its own, because addresses are assigned dynamically and may be reassigned between sessions. The Ethernet layer of frame 83601 gives a stronger identifier: the source hardware address 00:17:f2:e2:c0:ce, which Wireshark resolves to an Apple-registered OUI (Figure 9). Hardware addresses are bound to a physical interface and persist across address changes.

Filtering the entire capture on that hardware address returns 73,246 frames, 77.6% of the capture, totalling 44,742,268 bytes (Figures 11, 76 and 77). A single device therefore accounts for the substantial majority of activity in the capture, which both confirms the device was in sustained use during the capture window and gives a large body of traffic in which to look for identifying artefacts.

6.5 Attribution to a person

The decisive finding is that the same device leaked identity in traffic unrelated to the harassing message. Filtering the device’s traffic for HTTP cookies containing an "@" character, and then sweeping the verbose dissection with an email-address regular expression, recovered persistent

Google session cookies across 149 frames (Figures 12, 78 and 79). The recovered cookie pairs include:

- gmailchat = jcoachj@gmail.com/475090, repeated consistently throughout the session
- OL_SESSION = jcoachj@gmail.com-cal, associated with Google Calendar
- A form field req0_value carrying the same account identifier

These cookies were transmitted in cleartext by the same physical interface, within the same capture, that submitted the anonymous message. The anonymising service concealed the sender from the recipient; it did nothing to conceal the sender from anyone observing the network, because the user's ordinary browsing was authenticating to a personal account in parallel.

Figure 80 records the correlation in a single view: the form field email = lilytuckrige@yahoo.com and the spoofed sender the_whole_world_is_watching@nitroba.org appear alongside the gmailchat cookie for jcoachj@gmail.com. The same figure also shows an MSN Messenger login for cambridgecommons@hotmail.com from the same device, providing a second, independent account identifier that could be pursued as corroboration.

The chain of reasoning is therefore: the message was sent from 192.168.15.4 (network layer), by the interface 00:17:f2:e2:c0:ce (link layer), which was concurrently authenticated to jcoachj@gmail.com and cambridgecommons@hotmail.com (application layer). Each link is independently observable in the capture.

6.6 Limitations of the finding

Two qualifications should be recorded. First, the analysis identifies a device and the accounts in use on it; it does not by itself prove which person was operating the keyboard at 02:04:24. Establishing that would require corroboration outside the capture, such as dormitory access records or device custody. Second, hardware addresses can be changed in software. Nothing in this capture suggests that occurred, but the possibility should be acknowledged rather than assumed away.

6.7 Findings from the TShark exercises

The competence exercises produced a secondary but relevant finding about method. Every result obtained in the Wireshark interface during phase two was reproduced from the command line in phase four, and in several cases the command-line output was the more useful of the two.

Extracting the GET sequence with -T fields (Figure 73) produced a timeline in one command that would have required manual reading of the packet list in the interface. Counting the suspect device's frames with -qz io,stat (Figure 77) produced a byte total that the interface reports only as a percentage. Piping verbose output through a regular expression (Figures 78 and 79) searched 73,246 frames for identity artefacts in a single pass.

The graphical interface remains better for exploration, where the analyst does not yet know what to look for. Once the question is known, the command line answers it faster and leaves a reproducible record of exactly what was asked.

7. Challenges and Solutions

Several difficulties arose during the lab and are recorded here with their resolutions.

Malformed download command

The initial wget invocation was entered with the -q flag separated in a way the shell passed as a literal argument, and wget attempted to resolve the non-existent host "xn--q-5gn" before proceeding (Figure 1). The download subsequently completed correctly. The lesson is to read transfer output rather than assume success: had the real URL also failed, the error would have been easy to miss among the noise.

Volume of the evidence file

At 54 MB and 94,410 packets, nitroba.pcap cannot be reviewed frame by frame. The solution was to work top-down: identify the application-layer service first, then narrow to the specific transaction, then pivot to lower layers only once a specific frame was in hand. Attempting to start at the link layer would have produced 73,246 frames with no obvious way to rank them.

Uncertainty over valid filter field names

Several filter expressions initially returned nothing because the field name was guessed rather than confirmed. The solution was tshark -G fields, which prints the complete field dictionary and can be filtered with grep (Figures 52 to 63). Confirming that urlencoded-form.key and urlencoded-form.value existed before using them (Figure 75) was faster than repeatedly testing filters that silently matched nothing.

Regular expressions producing no or excessive matches

Patterns written directly at the command line frequently failed, either through shell quoting or through the expression itself being wrong. Two measures resolved this: composing and testing expressions in regex101.com before use (Figure 64), and testing patterns against a known sample string with echo before applying them to real output (Figure 49). Confirming that a pattern matches a string it should match is a quicker diagnosis than reasoning about why it did not match packet data.

Distinguishing capture filters from display filters

The two filter languages are not interchangeable: capture filters use BPF syntax and are applied irrevocably at collection time, while display filters use Wireshark syntax and are applied to an existing capture. Working through both deliberately — -f with BPF expressions in Figures 19 and 20, -Y with Wireshark expressions in Figures 31 to 33 — made the distinction concrete. The practical consequence is significant: an over-narrow capture filter discards evidence permanently, whereas an over-narrow display filter can simply be rewritten.

Verifying that filters behaved as intended

A filter that returns plausible output is not necessarily a correct filter. This was addressed by generating known traffic and checking that exactly that traffic appeared: ten pings producing ten captured echo replies (Figures 21 and 22), and a single browser page load producing a capture containing exactly the two expected hosts (Figures 65 to 68). Validating tooling against a known answer before applying it to an unknown one is a general forensic discipline, not merely a convenience.

8. Conclusion

The lab achieved both of its aims. The investigative objective was met: the anonymous harassing message sent to the complainant on 22 July 2008 was recovered in full from the packet capture, and was attributed first to a specific device on the dormitory network and then to a specific named account in use on that device. The competence objective was met in parallel, with the full range of TShark capabilities — capture, filtering, field extraction, statistics, stream reassembly and pattern matching — exercised and then applied to genuine evidence.

The central lesson is that anonymity at the application layer does not survive observation at the network layer when traffic is unencrypted. The sender took a deliberate step to conceal their identity by routing the message through an anonymising service, and that step was effective against the recipient. It was defeated within a few commands by an analyst with access to the network traffic, because the same device was simultaneously authenticating to personal accounts in cleartext. The identifying artefact was not in the harassing message at all; it was in unrelated browsing that happened to share a network interface.

A second lesson concerns evidential discipline. Hashing the file at acquisition and re-verifying it after analysis (Figures 2 and 69) cost two commands and established that the evidence was unchanged. Recording absolute timestamps rather than relative offsets (Figures 73 and 81) produced a timeline that means something outside the capture file. Neither step contributed to finding the answer, but both are what allow the answer to be relied upon.

Finally, the lab demonstrated the complementary relationship between graphical and command-line analysis. Wireshark answered the question "what is in this capture"; TShark answered "how much, when, and how often", and did so in a form that can be scripted, repeated and shown to someone else. Competent network forensics uses both.

9. Recommendations

Arising from the findings, the following recommendations are made.

For the institution in the scenario

- Require HTTPS across all institutional web services and enforce it with HSTS. The entire attribution in this investigation rested on cleartext HTTP; encryption would have prevented the leak of session cookies, and would equally protect legitimate users from credential theft on the same shared network.
- Segment the residential network and apply client isolation, so that devices on the dormitory network cannot observe one another’s traffic. The capture that solved this case would, in the hands of a malicious resident, have been a means of harvesting the session cookies of every other resident.

- Retain DHCP lease logs and switch-port MAC address tables for a defined period. Attribution in this case depended on a hardware address; without records tying that address to a registered device or port, the chain would have ended at the device rather than at a person.
- Publish a clear procedure for reporting harassment, including guidance to preserve message headers and timestamps, so that a complainant's report can be correlated with network records before those records age out.

For future analytical work

- Hash evidence at acquisition and re-verify after analysis as a matter of routine, and record both values in the report rather than only asserting that verification took place.
- Work from the application layer downwards when triaging a large capture. Identifying the service of interest first reduces the search space far more effectively than filtering on addresses.
- Pivot from IP address to hardware address early. IP addresses change within a capture; hardware addresses generally do not, and the pivot frequently reveals activity that address-based filtering misses.
- Search for identity in traffic adjacent to the incident, not only in the incident itself. In this case the incriminating artefact appeared in ordinary webmail and calendar traffic, not in the harassing message.
- Confirm field names with tshark -G fields before constructing filters, and test regular expressions against known sample strings before applying them to evidence. Both practices convert silent failures into immediate ones.
- Validate capture filters against deliberately generated traffic before relying on them. A capture filter discards data irrecoverably, so an error made at capture time cannot be corrected later.

10. References

Digital Corpora. (2008). Nitroba University Harassment Scenario. Retrieved from https://digitalcorpora.org/corpora/scenarios/nitroba-university-harassment-scenario/

Garfinkel, S. (2008). Nitroba University Harassment Scenario — nitroba.pcap [Data set]. Digital Corpora. Retrieved from https://digitalcorpora.s3.amazonaws.com/corpora/scenarios/2008-nitroba/nitroba.pcap

Wireshark Foundation. (n.d.). tshark(1) Manual Page. Retrieved from https://www.wireshark.org/docs/man-pages/tshark.html

Wireshark Foundation. (n.d.). Wireshark User's Guide. Retrieved from https://www.wireshark.org/docs/wsug_html/

Wireshark Foundation. (n.d.). Wireshark Display Filter Reference. Retrieved from https://www.wireshark.org/docs/dfref/

Firebug, J. (n.d.). regex101: build, test and debug regex. Retrieved from https://regex101.com/

National Institute of Standards and Technology. (2006). SP 800-86: Guide to Integrating Forensic Techniques into Incident Response. Gaithersburg, MD: NIST.

Offensive Security. (n.d.). Kali Linux Documentation. Retrieved from https://www.kali.org/docs/